M02 — Formularios, validación y Razor Pages en ASP.NET Core MVC

Cuaderno de prácticas

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Cómo trabajar con este cuaderno

En M01 el alumnado construyó un catálogo que responde a peticiones HTTP, utiliza rutas MVC, obtiene productos en un controlador y los muestra mediante vistas Razor y un layout. Durante M02 el problema cambia: el navegador debe poder enviar datos al servidor, recibir mensajes de validación y, finalmente, crear, editar y eliminar productos que se conservan en SQL Server Express LocalDB. La comparación final con Razor Pages mostrará otra manera de organizar la misma idea de entrada y salida, sin desarrollar una aplicación empresarial adicional.

El **recorrido obligatorio** se ajusta a las seis horas del segundo día. Las secciones de investigación, las aplicaciones autónomas y los retos son material de profundización que puede realizarse después de las demostraciones. No es necesario leer el manual entero en voz alta. Debe preservarse la relación entre cada modificación, el archivo que se cambia y la respuesta observable.

La organización consta de cinco prácticas: **P2.1**, HTML, GET, POST, model binding y antiforgery; **P2.2**, Tag Helpers de formularios; **P2.3**, DataAnnotations y ModelState; **P2.4**, CRUD con EF Core y LocalDB; **P2.5**, una Razor Page mínima. Cada punto utiliza una instantánea del proyecto situada en PROYECTO/2.x/CursoWeb.Mvc. Los proyectos de M01 no se modifican. Los estados de los ejercicios principales se conservan entre instantáneas; las variantes opcionales no tienen por qué venir activadas en las siguientes.

El trabajo del alumno distingue tres acciones: **observar** código ya activo para comprender su efecto; **activar** un bloque preparado y comentado que forma parte de un paso; y **escribir** una solución propia, sin disponer de su resultado ya implantado. Después de cada paso, hay que identificar la entrada, el componente responsable y la evidencia de salida. Las soluciones del manual se consultan solamente tras haber intentado resolver el enunciado.

Índice

- P2.1. El formulario HTML, el método HTTP, model binding y antiforgery.
- P2.2. Tag Helpers para campos, formularios y navegación.
- P2.3. Validación con DataAnnotations y ModelState.
- P2.4. Operaciones CRUD persistentes con EF Core y SQL Server Express LocalDB.
- P2.5. Una Razor Page: PageModel, OnGet y OnPost.

P2.1. Mostrar un formulario y recibir sus datos con MVC

Objetivo y punto de partida

Objetivo. Comprender una interacción completa de formulario: GET para mostrar controles, POST para enviar los campos, construcción de un objeto de entrada mediante model binding y respuesta HTML generada por una vista de confirmación. Observar, además, por qué un POST sin token antiforgery debe ser rechazado. La práctica **no añade persistencia**: el catálogo inicial sigue mostrando tres productos.

Teoría relacionada: capítulo 2.1 del manual teórico de M02. Conviene estudiar primero la diferencia entre método, ruta, campos enviados y datos del modelo. No hace falta aprender EF Core otra vez para esta actividad.

Proyecto a abrir: M02_FORMULARIOS_RAZOR/PROYECTO/2.1/CursoWeb.Mvc/CursoWeb.Mvc.csproj. Trabajaremos con Visual Studio y .NET 8 en Windows, aunque los ejemplos de compilación y navegación también pueden comprenderse con una terminal. No es necesario tener instalado LocalDB hasta P2.4.

Desde la raíz del repositorio, ejecuta:

```
cd M02_FORMULARIOS_RAZOR/PROYECTO/2.1/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Si Visual Studio inicia la aplicación por otro puerto, utiliza la dirección que muestra su consola o el perfil de ejecución. Para seguir los comandos exactamente, el ejemplo de este cuaderno utiliza el puerto 5091. Detén cualquier instancia anterior de MVC que siga usando esa dirección.

Mapa de trabajo de P2.1

Paso	Archivo	Acción del alumno	Evidencia observable
2.1.1	Controllers/ProductosController.cs y Views/Productos/Index.cshtml	Observar la aplicación heredada	GET /Productos muestra tres productos
2.1.2	Views/Productos/Crear.cshtml	Analizar el formulario entregado y sus campos	GET /Productos/Crear devuelve 200 y el formulario HTML

Paso	Archivo	Acción del alumno	Evidencia observable
2.1.3	Controllers/ProductosController.cs	Activar POST en el bloque PASO 2.1.3	Una petición protegida representa Confirmacion.cshtml
2.1.4	Crear.cshtml y acción POST	Comparar el envío normal con uno sin token	La solicitud sin token obtiene 400
2.1.5	Views/Productos/Confirmacion.cshtml	Escribir una condición Razor propia	Mostrar aviso solo si el precio es al menos 4
Reto 2.1	Confirmacion.cshtml	Activar y razonar una condición de longitud	Aviso solo si Nombre supera ocho caracteres

No ejecutes P2.2 sobre este proyecto: abre su instantánea cuando hayas terminado de comprender el resultado del POST.

Paso 2.1.1 — Confirmar que conservamos la aplicación del módulo anterior

Abre http://localhost:5091/ y comprueba que aparece la portada conocida. Visita después http://localhost:5091/Productos y verifica los nombres Cuaderno, Lápiz y Carpeta, con precios. Finalmente, abre http://localhost:5091/Productos/Detalle/2: la ficha debe mostrar Lápiz e identificador 2. No estamos reconstruyendo desde cero las vistas ni el enrutamiento del M01; partimos de una aplicación MVC navegable.

Abre el archivo Controllers/ProductosController.cs. Localiza ObtenerProductosEjemplo() y observa cómo se construye una lista fija de tres objetos. Compara ese método con Index(): la acción de catálogo solicita los productos y se los entrega a la vista. El navegador no accede directamente a la colección C# ni conoce cómo se creó.

Pregunta antes de cambiar código: ¿si más adelante una acción POST recibe un producto y devuelve su nombre en otra vista, eso hará que aparezca automáticamente como cuarto elemento en la lista? No. Recibir y presentar un objeto no lo incorpora a ObtenerProductosEjemplo(). Tampoco lo inserta en SQL Server. La persistencia será un paso diferente y explícito en P2.4.

Para comprobar el estado HTTP sin depender del estilo visual, puedes usar otra terminal:

```
curl -i http://localhost:5091/Productos
curl -i http://localhost:5091/Productos/Detalle/2
```

Confronta el resultado con lo que ves en el navegador. Una página bien renderizada no implica que se haya ejecutado una operación de escritura; de momento todas estas solicitudes son GET.

Paso 2.1.2 — Mostrar el formulario con GET

En ProductosController.cs localiza el método que prepara la nueva vista:

```
[HttpGet]
public IActionResult Crear() => View(new Producto());
```

Este método no recibe los datos del formulario. Su responsabilidad es **crear un modelo inicial para la página**. Un objeto Producto nuevo tendrá Nombre vacío y Precio igual a cero. Ese objeto se entrega al archivo Views/Productos/Crear.cshtml, el cual comienza declarando que trabaja con un único Producto mediante @model.

Abre Crear.cshtml y examina sus controles:

```
<form method="post" action="/Productos/Crear">
    @Html.AntiForgeryToken()
    <div>
        <label for="Nombre">Nombre</label>
        <input id="Nombre" name="Nombre" value="@Model.Nombre" />
    </div>
    <div>
        <label for="Precio">Precio</label>
        <input id="Precio" name="Precio" type="number" step="0.01"

value="@Model.Precio.ToString(System.Globalization.CultureInfo.InvariantCulture)" />
    </div>
    <button type="submit">Enviar producto</button>
</form>
```

Estudia la diferencia entre las propiedades del formulario. method="post" indica el método de la siguiente petición, no el de la solicitud que está representando ahora esta vista. action="/Productos/Crear" define adónde se enviará esa petición. name="Nombre" y name="Precio" son las claves que viajarán en el cuerpo de datos. El binder las buscará para asignarlas a las propiedades del objeto C#.

Por su parte, el atributo id identifica un elemento en el documento HTML; una etiqueta label puede referirse a él mediante for. En este ejemplo el id y el name utilizan cadenas iguales, pero no debemos confundir la asociación de etiquetas HTML con el enlace de datos del servidor. Las expresiones @Model.Nombre y

@Model.Precio ya se habrán evaluado antes de que el navegador reciba la respuesta.

Abre http://localhost:5091/Productos/Crear y comprueba la existencia de dos campos y del botón Enviar producto. En las herramientas de desarrollo (F12), selecciona Red e inspecciona la solicitud del documento: es **GET**, y el resultado esperado es **200**. Desde la pestaña Elementos busca el formulario y localiza un campo oculto con name="__RequestVerificationToken". Su valor lo genera ASP.NET Core; no lo copies ni lo utilices como texto fijo en el material.

Experimento negativo antes de activar el POST

Escribe Nombre = Rotulador y Precio = 7, pulsa Enviar y anota el método de la nueva petición. En este estado del proyecto, la acción POST sigue comentada. Por tanto, GET está disponible, pero POST no debe procesar el formulario: es esperable **405 Method Not Allowed**.

Explica por qué no se trata de una página HTML inexistente: has obtenido el formulario con GET hace un momento. Lo que falta es una acción seleccionable para este **otro método HTTP**. Si la confirmación aparece ya en este paso, revisa que estás abriendo la carpeta 2.1 y que no has activado el POST en una ejecución anterior.

Detén Kestrel antes de editar los comentarios del controlador.

Paso 2.1.3 — Activar el POST y examinar el model binding

En Controllers/ProductosController.cs localiza el bloque que empieza con **PASO 2.1.3 INICIO** y termina en **PASO 2.1.3 FIN**. Está dentro de un comentario C#. Elimina únicamente las marcas de apertura y cierre del comentario para que la acción se compile. Debes conservar ambos atributos:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Crear(Producto producto)
{
    return View("Confirmacion", producto);
}
```

No añadas instrucciones de persistencia, ni cambies la lista de productos, ni retires la protección antiforgery. El bloque de código preparado contiene

comentarios que explican el papel del binder: no son instrucciones para guardarlo todo en la base de datos.

Contrasta las dos acciones. Crear() GET no tiene parámetros y devuelve la vista del formulario con un objeto vacío. Crear(Producto producto) POST recibe un objeto cuyos valores provienen de los campos enviados por el navegador y devuelve la vista Confirmacion.cshtml. El framework elige la acción a partir de la ruta y el método HTTP; no está ejecutando dos métodos arbitrarios por el mero hecho de llamarse Crear.

Predicción: cuando el navegador envía Nombre=Rotulador y Precio=7 con un token válido, la segunda acción recibe un Producto con esos valores. Su Id sigue siendo cero, porque ese campo no se envía en el formulario. El binder construye un objeto, pero nadie lo guarda ni calcula un nuevo identificador de base de datos.

Compila de nuevo, ejecuta la aplicación y vuelve a cargar el formulario. Introduce Nombre = Rotulador y Precio = 7. Pulsa Enviar. Debe abrirse la página de confirmación con un encabezado Datos recibidos del formulario. Comprueba en su HTML que aparece el nombre Rotulador y el precio 7,00 o 7.00 según la cultura de presentación configurada.

Localiza Views/Productos/Confirmacion.cshtml. La vista recibe un único Producto y presenta sus propiedades mediante expresiones Razor. El resultado confirma que el binder obtuvo datos y que MVC los entregó a la vista. **No demuestra** que exista un cuarto producto en el catálogo. Regresa a /Productos y comprueba que la lista sigue mostrando exactamente Cuaderno, Lápiz y Carpeta.

Paso 2.1.4 — Comparar el POST legítimo con uno sin antiforgery

La vista Crear contiene @Html.AntiForgeryToken(), que inserta un campo oculto. El controlador utiliza [ValidateAntiForgeryToken], que exige comprobar el token al recibir la petición. Ambas partes intervienen en una misma protección. El token no es una regla que determine si el nombre o el precio son apropiados para el negocio; esa validación será el objeto de P2.3.

Primera observación, envío correcto: abre de nuevo /Productos/Crear y envía Nombre=Carpeta grande y Precio=9. En F12 → Red selecciona la petición POST y observa sus campos: Nombre, Precio y __RequestVerificationToken. Puedes inspeccionar la presencia del campo sin publicar su valor en un documento. La acción debe responder con la confirmación HTML cuando el envío es válido.

Segunda observación, solicitud sin token: desde otra terminal realiza un POST directo sin haber obtenido ni enviado el token. En PowerShell, escribe:

```
curl.exe -i -X POST http://localhost:5091/Productos/Crear --data
"Nombre=Rotulador&Precio=7"
```

El resultado esperado es **400 Bad Request** porque la acción requiere la validación antiforgery. No intentes obtener un 200 eliminando [ValidateAntiForgeryToken]; eso destruiría el sentido del experimento. Comprueba que el fallo se produce **antes** de procesar la operación del cuerpo.

Situación	Estado previsto	Qué debes concluir
GET /Productos/Crear	200	La ruta y la vista del formulario funcionan
POST antes de habilitar la acción	Normalmente 405	Falta el destino para el método POST
POST después de habilitar la acción pero sin token	400	La protección antiforgery impide procesar el envío
POST desde el formulario normal con token válido	200	Antiforgery, binder, controlador y vista intervienen correctamente

Si observas un 404, comprueba la ruta y el puerto. Si el POST sin token consigue un 200, confirma que mantienes el atributo del controlador y no estás enviando datos a otro endpoint. Si el POST desde el navegador obtiene 400, revisa la presencia del campo oculto, las cookies correspondientes y cualquier alteración del formulario. No supongas que todos los errores de entrada proceden del model binding.

Análisis: las responsabilidades permanecen separadas

Dibuja en papel la secuencia navegador → HTTP POST → antiforgery → model binding → acción Crear(Producto) → View(Confirmacion, producto) → HTML. Aclara que antiforgery y model binding son mecanismos del servidor ASP.NET Core, no servidores HTTP independientes. Identifica qué dato se transmite y en qué momento queda disponible como una propiedad decimal.

Explica también por qué esta acción es distinta de una Web API: hemos recibido campos de un formulario HTML y respondido con una vista Razor. Los conceptos HTTP fundamentales siguen siendo los mismos, pero la representación de salida es un documento HTML generado en el servidor.

Paso 2.1.5 — Aplicación autónoma: escribir Razor según un dato recibido

Hasta aquí has observado y activado bloques preparados. En este paso tendrás que **escribir** código propio. Abre Views/Productos/Confirmacion.cshtml y localiza el comentario **APLICACION 2.1.5**. Sustitúyelo por un condicional Razor con @if que muestre un elemento:

```
<strong id="precio-elevado">Importe elevado para el ejemplo</strong>
```

Ese elemento debe aparecer solamente cuando **Model.Precio >= 4m**. La letra m denota un literal decimal de C#, coherente con el tipo de la propiedad Precio. No cambies el controlador ni el modelo. El cambio solicitado es una regla de presentación que pertenece a la vista de confirmación.

Prueba dos entradas por separado. Envía primero Rotulador con precio 7 y verifica que aparece el texto de aviso. Regresa a Crear y envía Goma con precio 2: el elemento debe desaparecer. Una implementación que introduce la etiqueta sin condición pasaría la primera comprobación pero fallaría la segunda. Por eso hay que examinar tanto el caso positivo como el negativo.

Si el HTML no cambia, recuerda que un comentario Razor @* ... *@ no se representa en el documento. Asegúrate de haber escrito el bloque fuera del comentario. Si aparece siempre, comprueba dónde abriste y cerraste las llaves del @if. Si aparece nunca, revisa qué vista ejecuta la acción POST y cuál es el precio recibido.

Esta actividad no valida ningún precio real de negocio ni cambia la colección de productos. Su objetivo es poner en relación **la entrada de datos por POST** con **la presentación Razor** que ya conoces desde M01.

Solución de referencia, solo después de intentar el ejercicio

```
@if (Model.Precio >= 4m)
{
    <strong id="precio-elevado">Importe elevado para el ejemplo</strong>
}
```

La vista evalúa la expresión cada vez que MVC representa una respuesta de confirmación. No hay una variable JavaScript ni un enlace a una nueva acción: la condición se ejecuta en el servidor.

Reto 2.1 — Un segundo criterio visual independiente

El archivo Confirmacion.cshtml contiene un bloque marcado **RETO 2.1 INICIO / RETO 2.1 FIN**. En esta ocasión se entrega el código comentado y el alumno debe activarlo después de comprenderlo. El objetivo es mostrar el aviso nombre-extenso cuando la longitud del nombre recibido **supere ocho caracteres**.

Predice antes de ejecutarlo qué debe ocurrir si Nombre tiene exactamente ocho caracteres, nueve caracteres o está vacío. La condición preparada comprueba primero que el texto no sea nulo ni espacios en blanco y luego compara su longitud; ese orden evita consultar Length sobre un valor nulo.

Activa el bloque quitando sus delimitadores de comentario. Envía Rotulador, cuyo nombre supera ocho caracteres, y después Goma, que no los supera. Comprueba la aparición y ausencia de la etiqueta respectivamente.

```
@if (!string.IsNullOrWhiteSpace(Model.Nombre) && Model.Nombre.Length > 8)
{
    <p id="nombre-extenso">El nombre tiene más de ocho caracteres.</p>
}
```

El operador && utiliza evaluación de cortocircuito: la segunda condición solo se evalúa cuando la primera lo permite. El ejercicio no impide que lleguen nombres vacíos al servidor: solamente controla qué se dibuja en esta vista. En P2.3 estudiaremos cómo rechazar realmente entradas inválidas.

Si realizaste también la aplicación autónoma anterior, envía un nombre largo con precio 7 y otro corto con precio 2. Deben aparecer los avisos correspondientes sin afectar a los otros. La coexistencia de dos condiciones distintas sobre un mismo modelo demuestra que la vista puede presentar varias propiedades sin duplicar acciones del controlador.

Diagnóstico aplicado y transferencia a P2.2

Caso A: la vista Crear aparece, pero no se llega a Confirmación. Observa el estado HTTP. Un 405 indica que conviene revisar la acción POST; un 400 puede señalar antiforgery; un 500 requiere consultar la excepción de la aplicación. No «corrijas» todos esos casos modificando los campos de HTML sin examinar primero la respuesta.

Caso B: el nombre se representa correctamente pero el precio es incorrecto. Comprueba el campo name="Precio", el valor enviado realmente por el

navegador y la propiedad decimal del objeto. Si la entrada falla por cultura o por un formato incompatible, no alteres de manera arbitraria el valor de la entidad. La explicación completa de validación y conversión pertenece a P2.3.

Caso C: se ve una confirmación correcta pero al volver al catálogo no aparece el producto nuevo. Ese es precisamente el estado de P2.1: la acción devuelve View("Confirmacion", producto) y no guarda ni modifica la colección. Describe qué operaciones serán necesarias en P2.4: obtener DbContext, añadir entidad, guardar los cambios y volver a consultar el catálogo.

Caso D: el botón funciona solo cuando se quita el atributo ValidateAntiForgeryToken. La «solución» está debilitando el mecanismo de protección y por tanto no es una solución. Hay que examinar cómo se genera el campo oculto y se conserva la cookie antiforgery.

La meta de P2.1 es poder explicar el recorrido entero sin confundir entrada HTTP, enlace de modelos, selección de la acción, protección antiforgery y presentación Razor. En **P2.2** sustituiremos progresivamente atributos escritos a mano por Tag Helpers (asp-for, asp-action, asp-controller). Conservaremos el mismo formulario conceptual antes de estudiar la validación en P2.3 y la persistencia con LocalDB en P2.4.

Estado final del recorrido esencial: GET representa el formulario, POST protegido recibe Nombre y Precio y muestra una confirmación. El catálogo todavía contiene tres productos. Ese resultado es correcto y prepara la siguiente práctica.

Cuaderno de profundización opcional P2.1 — Rastrear el contrato completo

Esta actividad no añade acciones MVC ni obliga a cambiar el proyecto. Sirve para practicar la lectura de solicitudes reales una vez que hayas activado el bloque `PASO 2.1.3`. Utiliza el mismo proyecto `PROYECTO/2.1/CursoWeb.Mvc` y la misma dirección de `/Productos/Crear`. Mantén abierta la pestaña **Red** de las herramientas de desarrollo antes de pulsar el botón del formulario.

Primera observación: la petición que obtiene la página. Actualiza la dirección por GET y selecciona la solicitud del documento HTML. Identifica la URL y el método. En la respuesta, busca el elemento `form` y comprueba el destino declarado. Localiza después el campo oculto antiforgery; su presencia demuestra que el formulario está preparado para enviar el token, pero no demuestra todavía que se haya aceptado

ningún POST. Anota qué parte pertenece al HTML y qué parte pertenece a la solicitud HTTP, evitando describirlas como si fueran lo mismo.

Segunda observación: valores enviados. Escribe «Rotulador» y 7, pulsa Enviar y examina el POST. Compara los nombres reales de los campos con las propiedades del objeto de destino. Busca `Nombre` y `Precio` entre los datos de formulario. Si ves un control con un texto correcto en pantalla pero la propiedad correspondiente llega vacía, el primer diagnóstico consiste en revisar el `name`, no en revisar el formato visual de la etiqueta. Revisa la acción del controlador y justifica por qué el objeto de entrada se puede construir sin que cada campo aparezca como parámetro independiente del método.

Tercera observación: misma URL, distinto verbo. Vuelve a comentar temporalmente el bloque POST identificado en el proyecto y recompila. El GET deberá seguir ofreciendo el formulario; la operación POST, al no tener el endpoint adecuado, podrá rechazarse como método no permitido. Reactiva el bloque tal como estaba. Este cambio representa una transición explícita del ejercicio, no una modificación permanente del producto. Describe por qué un 405 no se corrige alterando la vista de confirmación y por qué no es equivalente a un 400 por protección antiforgery.

Cuarta observación: protección independiente del contenido. Repite el envío desde el navegador y compara la petición legítima con una petición fabricada sin token, como la que ya realizaste en el itinerario principal. La ausencia del token debe impedir alcanzar la acción protegida incluso si `Nombre` y `Precio` tienen valores válidos. Por tanto, *datos válidos* y *solicitud autorizada a llegar a la acción* son preguntas distintas. Si un 400 aparece también al pulsar el formulario normal, comprueba la cookie, el campo oculto y el destino de la petición antes de eliminar protecciones.

Conclusión razonada. Dibuja en tu cuaderno cuatro cajas: navegador, endpoint MVC, binder y vista de confirmación. Escribe junto a cada caja una evidencia concreta obtenida del navegador o del código. Debes poder explicar quién genera HTML, quién convierte valores, quién verifica el token y quién crea el objeto representado. El catálogo de productos fijos no debería contener el nombre que acabas de remitir: todavía no hemos introducido persistencia. Esta última consulta al catálogo es necesaria para no confundir «formulario enviado» con «entidad guardada».

Si algo falla. Antes de repetir todo el ejercicio, comprueba que el bloque 2.1.3 está activo, que has guardado el archivo y que la aplicación se ha recompilado. Usa un único puerto de Kestrel para evitar revisar respuestas antiguas de otro proceso. Un texto que aparece en una vista de confirmación no basta, por sí solo, para afirmar que el binder recibió todos los valores previstos; comprueba también los campos y el tipo del modelo. Consulta la teoría 2.1, apartado «Profundización aplicada: leer el contrato HTTP».

Resolución guiada P2.1 — Tres incidencias parecidas que requieren soluciones distintas

La siguiente secuencia se realiza sobre el estado posterior a activar `PASO 2.1.3`. No cambia el proyecto ni constituye una parte adicional obligatoria de los primeros cuarenta minutos de prácticas. Su propósito es enseñar a diagnosticar una aplicación que no se comporta como esperábamos, partiendo de observaciones reproducibles en el navegador.

Incidencia A: el botón aparece, pero el servidor no acepta el envío

Imagina que acabas de abrir `/Productos/Crear`, el formulario se representa correctamente y puedes escribir un nombre. Al pulsar Enviar aparece un error de método no permitido. Antes de cambiar una línea, identifica qué sabemos con certeza: el servidor ha sido capaz de devolver la vista mediante GET, de modo que al menos ese endpoint y el renderizado funcionan. No sabemos todavía si existe una acción que admita POST para la misma ruta. Ve a `Controllers/ProductosController.cs`, localiza las dos variantes de `Crear` y comprueba que la marcada `[HttpPost]` está activa. Si sigue encerrada entre las marcas del paso 2.1.3, actívala exactamente como indica el cuaderno, recompila y repite el envío.

No intentes arreglar un 405 cambiando el precio o reduciendo el nombre del producto. El error no significa que `ProductoFormulario` tenga valores inválidos; significa que no se ha seleccionado correctamente un endpoint para la combinación solicitada de ruta y verbo. **Comprobación positiva:** con el POST activo, los datos válidos y el token legítimo deben llegar a la vista de confirmación. **Comprobación negativa:** desactivar otra vez la acción POST debe permitir reproducir el estado inicial, sin necesidad de eliminar proyectos o crear controladores adicionales. Al terminar, deja la acción activa.

Incidencia B: el endpoint POST existe, pero un cliente fabricado recibe 400

Ahora suponemos que el envío normal del navegador sí funciona. Abre una terminal y reproduce el POST sin token que ya se estudió en 2.1.4. La solicitud contiene valores, pero no acredita que procedan del formulario generado en la sesión correcta. El rechazo debe producirse antes de que la acción permita una respuesta de confirmación. El objetivo de esta prueba no es conseguir que cualquier herramienta mande datos a toda costa: es demostrar que la protección del endpoint discrimina entre dos solicitudes que pueden llevar un nombre y un precio idénticos.

Si una persona propone quitar `[ValidateAntiForgeryToken]` porque «así deja de fallar», analiza la consecuencia: estaría eliminando precisamente el mecanismo que el ejercicio trata de observar. Lo correcto es mantener la protección y realizar el contraste controlado. La respuesta esperada es diferente para las dos peticiones: el navegador con token obtiene la confirmación; la petición externa sin token se rechaza. Describe esta diferencia en términos de confianza de la solicitud, no de reglas de negocio.

Incidencia C: se ve el nombre en el formulario, pero la confirmación no lo representa

Con el formulario normal abierto, inspecciona el HTML del campo Nombre y verifica que contiene el atributo `name` que el binder espera. En M01 ya utilizaste expresiones Razor para representar propiedades; aquí necesitas además que el **formulario envíe** valores bajo nombres acordados. Observa la solicitud del navegador y compara el par enviado con la propiedad del modelo. Si el nombre no viaja o tiene una clave diferente, la acción no puede obtenerlo automáticamente en la propiedad prevista, aunque visualmente el usuario haya escrito texto.

En este escenario, no modifiques primero la clase `Producto` ni la semilla del catálogo: todavía no hay ninguna operación persistente que pueda explicar el problema. Comprueba por este orden el formulario renderizado, el cuerpo enviado y los valores del objeto recibido por la acción. El diagnóstico estará completo cuando seas capaz de señalar el elemento concreto donde se pierde la correspondencia.

Hoja de interpretación para autoevaluación

Después de cada caso, escribe una respuesta de dos o tres frases que identifique: qué petición se envió, qué componente decidió el resultado y qué archivo permitiría investigar la causa. No basta responder «hay un problema en el formulario». Una explicación satisfactoria de A nombra la selección por método HTTP; una explicación

de B identifica la protección antiforgery; y una explicación de C relaciona el atributo `name` con el model binding. Si todas tus explicaciones proponen cambiar el mismo archivo, probablemente estás confundiendo capas.

Como último contraste, vuelve al listado de productos y comprueba que ninguno de los nombres utilizados en estas pruebas se convirtió en una nueva entidad. No estamos verificando EF Core todavía; estamos delimitando el contrato de entrada. Precisamente esa claridad permitirá comprender en 2.4 qué cambia cuando el controlador ejecuta una operación de almacenamiento real.

Extensión opcional de documentación. Anota en tu cuaderno una pequeña traza textual de un envío satisfactorio: «GET del formulario → HTML con nombres de campos y token → POST legítimo → binder → acción → vista de confirmación → HTML devuelto». Bajo cada flecha, coloca una evidencia obtenida de la vista o de la pestaña Red. Esta representación muestra causalidad y será reutilizable cuando analicemos la redirección después de guardar productos.

P2.2. Tag Helpers: construir formularios y enlaces sin duplicar cadenas de rutas

Qué vamos a conseguir y cómo se relaciona con P2.1

La primera práctica del módulo demostró que una vista podía ofrecer un formulario GET, enviar sus campos con POST y presentar el objeto recibido en una página de confirmación. También comprobamos que una solicitud POST sin el token antiforgery válido debe rechazarse. Ahora **mantendremos exactamente ese recorrido funcional** y cambiaremos la manera de escribir sus elementos Razor, introduciendo progresivamente los Tag Helpers de ASP.NET Core MVC.

Esta decisión es deliberada. Cuando se aprende una técnica nueva, cambiar a la vez el modelo, la acción, las rutas, el HTML y la base de datos dificulta identificar por qué algo funciona o falla. En este punto el único componente que vamos a transformar es la **vista**. Conservaremos la misma acción `ProductosController.Crear` y los mismos campos `Nombre` y `Precio`; todavía no guardaremos registros en SQL Server.

Objetivos concretos: reconocer el papel de `_ViewImports.cshtml`; sustituir un `form` HTML por un Form Tag Helper; generar las asociaciones de `label` e `input` mediante `asp-for`; utilizar Anchor Tag Helpers para la navegación; comprobar que el token antiforgery y el model binding siguen funcionando; y aplicar las mismas ideas a enlaces de detalle cuyo identificador proviene de un objeto del catálogo.

Teoría relacionada: apartado 2.2 del manual teórico del M02, especialmente la distinción entre CSHTML y HTML final, los Tag Helpers incorporados, los atributos generados y el diagnóstico de problemas de enrutamiento.

Proyecto que debes abrir: `M02_FORMULARIOS_RAZOR/PROYECTO/2.2/CursoWeb.Mvc/CursoWeb.Mvc.csproj`. Es una instantánea del recorrido central de P2.1; la acción POST ya está activa. No hay que copiar a mano los cambios de otras carpetas ni modificar los proyectos de M01.

El recorrido esencial se compone de los pasos **2.2.1 a 2.2.5**. La aplicación autónoma y el reto final permiten practicar el razonamiento sin consumir tiempo del resto de la jornada. Todas las comprobaciones pueden realizarse desde el navegador y Visual Studio.

Preparar el entorno

Desde la raíz del repositorio puedes iniciar la aplicación con estos comandos:

```
cd M02_FORMULARIOS_RAZOR/PROYECTO/2.2/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Si existe otro proyecto ejecutándose en el mismo puerto, detenlo antes. Si trabajas desde el perfil de Visual Studio, utiliza la dirección que muestre el navegador o la consola de ejecución. Para que las comprobaciones sean reproducibles, los ejemplos siguientes usan `http://localhost:5091`.

Comprueba primero tres direcciones: `/`, `/Productos` y `/Productos/Crear`. La portada, el catálogo y el formulario deben abrirse. Envía **Nombre = Rotulador, Precio = 7** y verifica que la confirmación muestra los datos, sin añadir un cuarto producto al catálogo. Ese comportamiento es la **línea base** que no debemos alterar.

Mapa de pasos, archivos y comprobaciones

Paso	Archivo del proyecto	Trabajo y resultado esperado
2.2.1	`Views/_ViewImports.cshtml` y `Views/Productos/Crear.cshtml`	**Observar.** Identificar dónde se habilitan los Tag Helpers y cómo queda el HTML original
2.2.2	`Views/Productos/Crear.cshtml`	**Sustituir** la apertura del formulario y retirar el token manual. Conservar un token generado automáticamente
2.2.3	`Views/Productos/Crear.cshtml`	**Sustituir** los dos `label`. Verificar `for` para Nombre y Precio
2.2.4	`Views/Productos/Crear.cshtml`	**Sustituir** los dos `input`. Verificar `name`, `id`, `type` y envío de datos
2.2.5	`Views/Productos/Crear.cshtml`	**Sustituir** el enlace de regreso al catálogo. Comprobar su `href`
Aplicación autónoma 2.2.6	`Views/Shared/_TarjetaProducto.cshtml`	**Escribir** un Anchor Tag Helper para el producto de la tarjeta
Reto 2.2	`Views/Productos/Index.cshtml`	**Activar y explicar** tres enlaces dinámicos con `asp-route-id`

Los comentarios del proyecto contienen los identificadores exactos `PASO 2.2.2` a `PASO 2.2.5`, `APLICACION 2.2.6` y `RETO 2.2`. En los pasos guiados se presenta una alternativa

comentada: el alumno debe **sustituir el elemento original**, no añadir otra copia al lado.

Paso 2.2.1 — Observar cómo Razor transforma el marcado

Abre `Views/_ViewImports.cshtml` y encuentra estas líneas:

```
@using CursoWeb.Mvc
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

La segunda directiva hace disponibles los Tag Helpers integrados en el ámbito de las vistas. El archivo `Crear.cshtml` se encuentra debajo de `Views/Productos`, por lo que puede utilizarlos. Esta directiva no establece una ruta HTTP ni añade una acción al controlador; interviene cuando Razor procesa los elementos del archivo CSHTML.

En `Views/Productos/Crear.cshtml` localiza la apertura del formulario original:

```
<form method="post" action="/Productos/Crear">
    @Html.AntiForgeryToken()
```

Aquí `method` y `action` son atributos HTML ordinarios. La llamada `Html.AntiForgeryToken()`, en cambio, es un HTML Helper ejecutado por Razor en el servidor. El navegador no ejecuta ese método C#: recibe su **resultado**, que incluye un elemento oculto.

Abre `/Productos/Crear` y utiliza F12 → **Elementos** para encontrar la etiqueta `form`. Identifica su `action`, su `method` y el campo oculto con `name="__RequestVerificationToken"`. Localiza también los controles Nombre y Precio. No copies el valor real del token en el material de clase ni lo confundas con un dato permanente.

Predicción previa: ¿qué cambiará en la pantalla al utilizar Form Tag Helper? Posiblemente nada. Nuestro objetivo no es rediseñar la interfaz. Esperamos que el servidor genere un HTML funcionalmente equivalente, pero que el código Razor describa el destino por su controlador y su acción.

Una segunda predicción es más importante: si algún atributo `asp-controller` apareciera literalmente en el HTML final, ¿qué significaría? Que el Tag Helper no habría intervenido como esperamos. La primera comprobación sería el ámbito de `_ViewImports.cshtml` y la forma de escribir el elemento, no modificar al azar `ProductosController`.

Paso 2.2.2 — Form Tag Helper y continuidad de antiforgery

En `Crear.cshtml` busca **PASO 2.2.2**. Sustituye la apertura del formulario por:

```
<form asp-controller="Productos" asp-action="Crear" method="post">
```

Retira también la línea `@Html.AntiForgeryToken()`, que se utilizaba en el formulario HTML manual. Con Form Tag Helper integrado y el método POST, ASP.NET Core genera la entrada antiforgery correspondiente. La protección no debe desaparecer: ahora la genera otro mecanismo.

Este ejercicio exige precisión: se sustituye una apertura de formulario, se mantiene el contenido interior y se conserva exactamente un cierre `</form>`. No es correcto acabar con dos formularios ni con dos tokens generados por mecanismos distintos.

Detén la aplicación, compila y ejecútala otra vez. Abre `/Productos/Crear` y examina el HTML desde las herramientas de desarrollo. Comprueba que la etiqueta final tiene un `action` relacionado con `/Productos/Crear`, un `method="post"` y **un campo oculto** `__RequestVerificationToken`. Los atributos especiales `asp-controller` y `asp-action` no deberían aparecer sin procesar en el documento final.

Envía **Rotulador, 7**. La confirmación debe aparecer como antes; el controlador no ha cambiado. Después repite la prueba negativa enviando un POST sin token desde PowerShell:

```
curl.exe -i -X POST http://localhost:5091/Productos/Crear --data
"Nombre=Rotulador&Precio=7"
```

El servidor debe devolver **400 Bad Request** porque la acción mantiene `[ValidateAntiForgeryToken]`. Si una petición del formulario real devuelve 200 y otra sin token obtiene 400, acabamos de demostrar que el cambio de sintaxis no ha eliminado la protección.

Diagnóstico de un formulario que dejó de enviar

Un error habitual consiste en quitar `Html.AntiForgeryToken()` pero dejar activo el formulario original sin el nuevo Form Tag Helper. En ese caso falta la generación del campo oculto, y es razonable que la acción protegida rechace el envío normal. Para diagnosticarlo, comprueba primero el HTML recibido por el navegador y después los campos efectivamente transmitidos en F12 → Red.

No resuelvas un 400 quitando `[ValidateAntiForgeryToken]`. Eso desactivaría una protección y ocultaría la causa. Tampoco confundas un 405 con un 400: el primero puede indicar que no existe una acción POST compatible, mientras que el segundo puede proceder de antiforgery. Son hipótesis distintas que se verifican mirando las solicitudes y el código.

Paso 2.2.3 — Label Tag Helper y asociaciones accesibles

Localiza ambos comentarios **PASO 2.2.3** en `Crear.cshtml`. Sustituye los dos elementos `label` por:

```
<label asp-for="Nombre"></label>
<label asp-for="Precio"></label>
```

Cada uno debe permanecer junto a su campo. **Todavía no cambies los inputs**, porque queremos observar el efecto del Label Tag Helper de forma aislada.

Después de compilar, inspecciona las etiquetas generadas. Deben producir asociaciones como `for="Nombre"` y `for="Precio"`. Compara esos valores con los `id` existentes en los inputs. Si la asociación es correcta, seleccionar el texto de la etiqueta enfoca el campo correspondiente. Esa relación también aporta información a las tecnologías de asistencia.

La propiedad `Nombre` del modelo no se convierte en una variable JavaScript por usar `asp-for`. El Tag Helper utiliza los metadatos de la vista tipada durante el renderizado. Los campos del formulario siguen enviándose con las mismas claves, y la acción del controlador continúa siendo la misma.

En P2.3 estudiaremos cómo utilizar atributos como `[Display]`, `[Required]` y `[StringLength]` para describir los datos que queremos aceptar. Aquí no necesitamos adelantar esas reglas: basta con comprender **cómo se vincula una etiqueta con una propiedad del modelo**.

Paso 2.2.4 — Input Tag Helper y contrato de model binding

Busca los dos comentarios **PASO 2.2.4** y reemplaza los inputs originales por sus alternativas:

```
<input asp-for="Nombre" />
<input asp-for="Precio" type="number" step="0.01" />
```

No dejes los controles originales junto a los nuevos. Eso podría introducir dos entradas con el mismo `name` y dificultar la interpretación de lo que el navegador envía.

Compila y abre nuevamente el formulario. El resultado HTML debe contener los atributos `name` e `id` apropiados para `Nombre` y `Precio`. El campo Precio conserva el control numérico y el paso de centésimas que hemos indicado explícitamente. El atributo `asp-for` no debe aparecer como parte de la representación HTML final.

Envía primero **Rotulador, 7**. Comprueba que la acción POST recibe el dato y que la vista Confirmacion lo representa. Regresa al catálogo: los tres productos iniciales deben seguir existiendo y no debe haber un producto «Rotulador» añadido automáticamente.

Esta observación demuestra dos ideas complementarias. Por una parte, el Input Tag Helper ha generado los nombres necesarios para el enlace de modelos. Por otra, **nada de esto ha implementado persistencia** ni validación completa. `type="number"` es una indicación útil para la interfaz del navegador; las reglas de aceptación de datos en el servidor serán responsabilidad de P2.3.

Investigar un precio que no llega correctamente

Si el POST funciona pero el precio recibido no coincide con lo escrito, no modifiques a ciegas el controlador. Comprueba el `name` generado, el valor transmitido en F12 → Red y la cultura utilizada para la conversión. Para el recorrido básico utilizamos un precio entero, 7, precisamente para no añadir el problema independiente de los separadores decimales de distintas culturas.

El hecho de utilizar un Input Tag Helper no transforma un campo inválido en un dato correcto. El model binder puede encontrar problemas de conversión, y la acción debería comprobar `ModelState` cuando se incorporen las reglas de P2.3. **No presentes P2.2 como un formulario de producción plenamente validado**.

Paso 2.2.5 — Anchor Tag Helper para volver al catálogo

En `Crear.cshtml`, sustituye el enlace original por la alternativa del comentario **PASO 2.2.5**:

```
<p><a asp-controller="Productos" asp-action="Index">Volver al catálogo</a></p>
```

Inspecciona el HTML generado. El navegador debe recibir un elemento `a` con un `href` adecuado para la lista `/Productos`. Haz clic y comprueba que recuperas el catálogo.

Reflexiona sobre un posible malentendido: tanto la etiqueta `form` como la etiqueta `a` pueden utilizar `asp-action`, pero **no hacen la misma operación**. El formulario se envía con el método indicado, POST en nuestro caso. Un enlace ordinario inicia una nueva navegación GET cuando el usuario lo selecciona.

Con ello queda completado el itinerario esencial de P2.2. El alumno ha mantenido el mismo contrato HTTP y ha mejorado la relación entre el HTML de la vista, las propiedades del modelo y las acciones MVC. Antes de pasar a la ampliación, verifica conjuntamente los elementos principales: el `form` tiene destino correcto, el token existe una vez, las etiquetas están asociadas con los campos, el POST protegido funciona y el enlace regresa al catálogo.

Aplicación autónoma 2.2.6 — Escribir un enlace dentro de una parcial compartida

El siguiente ejercicio **exige escribir código**, no activar una solución preparada. En `Views/Shared/_TarjetaProducto.cshtml` encontrarás el comentario **APLICACION 2.2.6**, junto al enlace que actualmente utiliza una URL escrita a mano:

```
<a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
```

Debes sustituirlo por un Anchor Tag Helper propio que indique `Productos` como controlador, `Detalle` como acción y suministre el identificador de `Model.Id` mediante `asp-route-id`. Conserva el texto «Ver detalle de» y el nombre del producto.

Predice antes de ejecutar: la parcial es reutilizada por el catálogo de tarjetas y por la ficha individual. Por tanto, modificar **un solo archivo** debe afectar a los enlaces que se generan en ambos lugares. La prueba no consiste únicamente en que el enlace se vea subrayado; hay que comprobar que conduce a la ficha del producto correcto.

Abre `/Productos`, inspecciona la primera tarjeta y comprueba que su enlace conduce a la ficha del producto 1. Repite con las tarjetas 2 y 3. Después abre `/Productos/Detalle/2` y comprueba que la tarjeta insertada en esa página sigue enlazando al detalle de Lápiz.

No introduzcas un enlace fijo con identificador 2 dentro de la parcial. La misma vista se utiliza para varios objetos y la ruta debe depender de `Model.Id`.

Solución de referencia, solo después de intentarlo

```
<a asp-controller="Productos"
   asp-action="Detalle"
   asp-route-id="@Model.Id">Ver detalle de @Model.Nombre</a>
```

El resultado que debe llegar al navegador es un enlace HTML con un `href` generado a partir de la configuración de routing de MVC. `asp-route-id` suministra el valor del parámetro de ruta denominado `id`; no es un atributo HTML estándar ni guarda información en la base de datos.

Reto 2.2 — Enlaces dinámicos en el listado principal

En `Views/Productos/Index.cshtml` localiza **RETO 2.2**. El archivo contiene un enlace literal dentro del bucle `foreach`, y una alternativa comentada para reemplazarlo. El reto consiste en activar la alternativa y retirar el enlace anterior.

La alternativa debe expresar el controlador Productos, la acción Detalle y el valor de ruta `id` tomado de `producto.Id`. Como el catálogo tiene tres objetos con identificadores distintos, deben generarse **tres enlaces diferentes**.

```
<a class="enlace-detalle"
   asp-controller="Productos"
   asp-action="Detalle"
   asp-route-id="@producto.Id">Ver detalle</a>
```

Después de editar, cuenta los enlaces «Ver detalle» dentro del listado: debe existir exactamente uno por elemento. Si aparecen dos por producto, has añadido el nuevo sin retirar el anterior. Si los tres enlazan a la misma ficha, revisa que hayas usado `producto.Id` dentro del bucle en lugar de un número constante.

Navega a cada ficha desde el listado y comprueba que los identificadores, nombres y precios corresponden a Cuaderno, Lápiz y Carpeta. Regresa después al formulario de creación y comprueba que aún admite el POST protegido. Esas comprobaciones muestran que el nuevo uso de Anchor Tag Helpers no ha roto la funcionalidad que ya conocíamos.

Diagnóstico final y continuidad hacia P2.3

Cuando una página se ve correctamente pero un formulario deja de funcionar, conviene dividir el problema por responsabilidades. Si el botón de envío devuelve 405, inspecciona la ruta y la acción POST disponible. Si devuelve 400, busca el token y su cookie antes de cambiar reglas del modelo. Si devuelve una confirmación con campos vacíos, investiga los atributos `name` generados. Si el enlace al catálogo no funciona, inspecciona el `href` final y su ruta.

Estas comprobaciones no requieren modificar cuatro archivos simultáneamente. La propia progresión de P2.2 —formulario, labels, inputs y enlaces— ayuda a identificar cuál de las transformaciones introdujo un resultado inesperado.

Resultado de aprendizaje: sabemos generar formularios y enlaces mediante las capacidades integradas de MVC, pero aún no hemos definido qué datos debe aceptar el servidor. En **P2.3** crearemos un modelo de entrada específico, declararemos reglas mediante DataAnnotations y utilizaremos ModelState para mostrar errores sin perder los valores enviados. En **P2.4** añadiremos las operaciones de persistencia con SQL Server Express LocalDB y Entity Framework Core. La introducción a Razor Pages quedará para **P2.5**.

El formulario de P2.2 termina siendo más mantenible, pero la responsabilidad de aceptar datos válidos continúa estando en el servidor y no en la apariencia del HTML.

Cuaderno de profundización opcional P2.2 — Comparación guiada de HTML y Tag Helpers

Trabaja sobre `PROYECTO/2.2/CursoWeb.Mvc` tras completar los pasos 2.2.1 a 2.2.5. El objetivo es practicar una habilidad diferente de la escritura de sintaxis: **leer el resultado que el servidor entrega al navegador y justificarlo a partir de la vista Razor**. No es necesario crear nuevos archivos ni alterar rutas fuera de las preparadas.

Analiza el formulario. Abre `Views/Productos/Crear.cshtml` y localiza el Form Tag Helper. Escribe cuál es la acción lógica indicada en Razor y comprueba en el navegador cuál es el valor final de `action`. En la respuesta HTTP no debe quedar el atributo `asp-action` sin procesar. Observa si la ruta construida coincide con el controlador y acción activos. Explica por qué los atributos de Tag Helper son instrucciones de servidor y no una característica que el navegador tenga que entender.

Analiza las etiquetas de entrada. Para cada campo, localiza `asp-for` en la fuente Razor y compara la etiqueta `label` y el `input` generados. El vínculo debe permitir que seleccionar el texto visible de la etiqueta sitúe el foco en el control correspondiente. Examina `id` y `name` por separado. Si únicamente cambias el texto visible de la etiqueta, no deberías modificar el nombre del dato que se envía por POST. Si cambias el contrato de modelo, en cambio, la expresión de `asp-for` debe seguir apuntando a la propiedad correcta.

Analiza un enlace dinámico. En la vista parcial `Views/Shared/_TarjetaProducto.cshtml`, localiza el enlace al detalle que preparaste en la aplicación autónoma 2.2.6. Comprueba que el producto mostrado en esa tarjeta determina el valor usado para `asp-route-id`. Visita la ficha del primer producto y la del segundo desde sus tarjetas. Si dos enlaces llevan al mismo Id por error, investiga qué modelo recibió la parcial antes de cambiar el routing global. Repite la observación en `Views/Productos/Index.cshtml` para el reto 2.2 y comprueba que los enlaces de diferentes filas no comparten accidentalmente el mismo identificador.

Contrasta una ventaja y un límite. Un atributo `asp-controller` puede hacer más explícito el destino MVC y mantener el enlace integrado con routing. Sin embargo, escribir un nombre de controlador incorrecto sigue siendo un error posible. De manera parecida, `asp-for` puede generar los atributos de entrada, pero no valida automáticamente cualquier dato que llegue a la acción. El criterio útil es identificar *qué problema concreto de mantenimiento o expresividad* resuelve cada helper y qué responsabilidades permanecen en el modelo y el controlador.

Resultado esperado. Tu descripción debería relacionar un fragmento Razor con el HTML resultante, el dato de modelo utilizado y la solicitud o navegación observable. No basta una lista de nombres de helpers. Si la pantalla muestra atributos `asp-for` o `asp-route-id` literalmente, revisa `_ViewImports.cshtml` y la configuración del proyecto; si la navegación genera una URL equivocada, inspecciona el destino y el Id recibido, no el estilo CSS.

Transferencia a P2.3. En el punto siguiente veremos que los metadatos de validación pueden influir en el marcado de los campos, pero el servidor sigue siendo el responsable de decidir si acepta la operación. Esa relación se entenderá mejor porque ahora sabes diferenciar el código que escribió el desarrollador del HTML que realmente entrega ASP.NET Core.

Resolución guiada P2.2 — Investigar una interfaz después de convertirla a Tag Helpers

El proyecto del capítulo 2.2 permite comprobar que un formulario escrito con atributos de Tag Helpers sigue respondiendo al mismo contrato MVC. Esta propuesta de profundización evita añadir código nuevo: vuelve a utilizar los archivos que ya has trabajado en los pasos 2.2.2 a 2.2.5, la vista parcial del ejercicio autónomo y el reto del listado principal.

Incidencia A: un atributo de Razor aparece sin transformar en el navegador

Supongamos que inspeccionas el HTML recibido y encuentras una etiqueta cuyo atributo todavía se llama `asp-for` o `asp-action`. No es el resultado que se espera cuando el Tag Helper correspondiente se aplica. Abre `Views/_ViewImports.cshtml` y confirma que el proyecto habilita los Tag Helpers de MVC. Después comprueba que estás viendo la respuesta del proceso actualizado y no el código fuente `.cshtml` abierto directamente como archivo local. Un navegador no sabe interpretar los atributos `asp-*` por sí mismo; necesita el HTML producido por el servidor ASP.NET Core.

En tu explicación, separa dos causas que suelen mezclarse: una cosa es que Razor no haya aplicado un Tag Helper y otra que lo haya aplicado pero haya generado un destino incorrecto. En el primer caso inspeccionamos importación y configuración; en el segundo, la acción, el controlador y las convenciones de routing. Ambos síntomas pueden afectar a un formulario, pero exigen comprobaciones diferentes.

Incidencia B: el formulario sigue funcionando, pero su botón de regreso apunta a otra ubicación

Revisa la versión del enlace creada con `asp-controller` y `asp-action`. Abre el resultado HTML para obtener el destino efectivo. Haz clic y comprueba si regresaste al catálogo MVC esperado. A continuación, utiliza la vista de detalle y regresa de nuevo al listado. Este pequeño recorrido muestra que generar URLs a partir de nombres de acciones reduce la cantidad de rutas literales que escribimos, sin eliminar la necesidad de comprobar la navegación resultante.

Piensa qué cambiaría si el catálogo se publicase bajo una ruta distinta. No hace falta implementar ese cambio para contestar: con un enlace literal habría que localizar las cadenas afectadas; con un enlace construido desde acciones dependeríamos de que esas acciones y rutas siguieran registradas correctamente. Una solución que

evita duplicar cadenas puede ser más mantenible, pero no valida automáticamente un nombre mal escrito.

Incidencia C: todas las tarjetas muestran un texto distinto, pero sus enlaces llevan a la misma ficha

La vista parcial `_TarjetaProducto.cshtml` recibe un producto. Abre su código y explica de dónde procede `Model.Id` cuando un listado llama a esa parcial varias veces. Después inspecciona los enlaces renderizados en dos tarjetas distintas y compara sus direcciones. Si el identificador fuera constante, la interfaz estaría mostrando datos distintos con un destino común. El problema tendría relación con el valor utilizado en la vista parcial, no con que el navegador «no entienda Tag Helpers».

En el reto 2.2 también activaste enlaces en `Views/Productos/Index.cshtml`. El mismo criterio permite comprobarlos: cada iteración necesita utilizar el Id del producto que representa esa fila. Contrasta un enlace de tarjeta con otro del listado. Deben conducir a la misma ficha si representan el mismo elemento; deben conducir a fichas distintas cuando los Id son distintos. Esta comparación es una prueba conceptual de reutilización, no una nueva implementación de la operación Detalle.

Incidencia D: la pantalla tiene un campo nuevo, pero el dato no llega al servidor

Imagina un campo visible añadido manualmente con un nombre que no pertenece a `ProductoFormulario`. El navegador puede enviarlo como texto, pero eso no crea por sí solo una propiedad C# ni modifica el contrato de una acción. Lo importante es que la expresión `asp-for` se refiere a una propiedad del modelo de la vista; su utilidad procede de declarar explícitamente el vínculo. No añadas un campo extra en esta práctica: simplemente observa el contrato ya existente y explica por qué no podemos asumir que cualquier entrada se convertirá en una propiedad del objeto de destino.

Conecta esta conclusión con la validación del capítulo siguiente. Los Tag Helpers facilitan la escritura del marcado y pueden generar metadatos a partir del modelo, pero no son una alternativa a comprobar el contenido de una solicitud. Un formulario que luce profesional y utiliza Tag Helpers puede seguir necesitando reglas de longitud, rango y obligatoriedad.

Tabla de decisiones a completar

Prepara cuatro filas en tu cuaderno: formulario sin transformación, URL errónea, identificador de tarjeta incorrecto y campo sin propiedad equivalente. Para cada

una, anota qué archivo inspeccionarías primero, qué mirarías en HTML y qué debería mantenerse intacto. Una respuesta razonable relacionará las incidencias con `_ViewImports.cshtml`, las expresiones de enlaces, el contexto de la parcial y el contrato de entrada respectivamente.

Al terminar, repite el POST normal de creación. Debe seguir presentando los valores recibidos; la conversión a Tag Helpers no ha introducido persistencia. Este control final evita otro error de aprendizaje frecuente: creer que sustituir una etiqueta HTML por su variante tipada significa haber cambiado la arquitectura de la aplicación.

P2.3. Validar los datos del formulario y representar sus errores

Objetivo y relación con las prácticas anteriores

Hasta ahora el alumno ha conseguido obtener un formulario mediante GET, enviarlo con un POST protegido y representar la respuesta mediante Razor. Además, en P2.2 ha sustituido las asociaciones HTML escritas a mano por Tag Helpers. Nuestro siguiente problema es diferente: **no basta con que el navegador envíe datos y el servidor sepa recibirlos; deben cumplir unas reglas antes de convertirse en un producto válido**.

En P2.3 aprenderás a separar el contrato de entrada del modelo que representa la entidad; activarás reglas DataAnnotations, investigarás cómo registra MVC los errores de conversión y validación en `ModelState`, mostrarás mensajes junto a los campos sin borrar lo que ya había escrito el usuario y compararás la validación de servidor con la anticipada de cliente. Mantendremos expresamente **desactivada la persistencia**: en el catálogo seguirán Cuaderno, Lápiz y Carpeta, sin un cuarto elemento nuevo.

El recorrido troncal de esta práctica comprende los pasos **2.3.1 a 2.3.4**. La activación de scripts en 2.3.5 se hará como demostración breve, cuando la conectividad del laboratorio lo permita; el ejercicio de presentación 2.3.6 y el reto son ampliaciones que pueden completarse después. Así respetamos el horario del día sin renunciar a que el manual sea un recurso de aprendizaje profundo.

Teoría relacionada: punto 2.3 del manual del módulo. Si un resultado sorprende, vuelve a consultar la distinción entre model binding, validación declarativa y visualización de errores.

Proyecto a abrir: `M02_FORMULARIOS_RAZOR/PROYECTO/2.3/CursoWeb.Mvc/CursoWeb.Mvc.csproj`. No modifiques el M01 aprobado ni las carpetas de 2.1 o 2.2. La carpeta 2.3 es el punto de partida propio de este ejercicio y conserva los mecanismos HTML y Tag Helpers estudiados antes.

Arranque reproducible

Desde la raíz del repositorio:

```
cd M02_FORMULARIOS_RAZOR/PROYECTO/2.3/CursoWeb.Mvc
dotnet restore
```

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Detén cualquier aplicación anterior que siga ocupando el puerto 5091. Abre `http://localhost:5091/Productos/Crear`; debe mostrarse el formulario con dos campos, «Nombre» y «Precio», y el botón **Validar producto**. Comprueba también `/Productos`, donde aparecen los mismos tres productos del módulo anterior.

No necesitas SQL Server Express LocalDB para esta práctica. Aunque estudiaremos qué ocurre con una entrada inválida, no habrá ninguna escritura persistente que se deba deshacer: esta separación permite investigar el formulario sin introducir una segunda fuente de errores.

Correspondencia completa entre pasos, archivos y resultados

Paso	Archivo concreto	Intervención	Comprobación prevista
2.3.1	`Models/ProductoFormulario.cs`, `Controllers/ProductosController.cs`, `Views/Productos/Crear.cshtml`	Observar la separación entre entrada y entidad	No llega Id desde el formulario; la confirmación sigue disponible
2.3.2	`Models/ProductoFormulario.cs`	Descomentar cuatro DataAnnotations	Los valores ausentes, demasiado cortos y fuera de rango producen errores
2.3.3	`Controllers/ProductosController.cs`	Inspeccionar `ModelState.IsValid`; activar diagnóstico comentado opcional	Entrada incorrecta vuelve a Crear; en la depuración se identifican campos
2.3.4	`Views/Productos/Crear.cshtml`	Observar `asp-validation-for` y comparar `ModelOnly` con `All`	Mensajes visibles junto al campo, valor válido conservado
2.3.5	`Crear.cshtml`, `_Layout.cshtml`, `_ValidationScriptsPartial.cshtml`	Activar una sección Razor comentada	Validación cliente opcional; la del servidor sigue funcionando
2.3.6	`Models/ProductoFormulario.cs`, `Crear.cshtml`	Escribir `Display(Name=...)` en el modelo de entrada	La etiqueta cambia sin cambiar la clave HTTP `Nombre`
Reto 2.3	`Controllers/ProductosController.cs`	Activar un bloque `ModelState.AddModelError`	Nombres que comienzan por TEST se rechazan aunque tengan longitud válida

Los comentarios `PASO 2.3.2`, `PASO 2.3.3`, `PASO 2.3.4`, `PASO 2.3.5`, `APLICACION 2.3.6` y `RETO 2.3` permiten localizar directamente el código correspondiente. Cada intervención tiene

su propia comprobación observable. No actives todos los comentarios simultáneamente antes de observar sus efectos.

Paso 2.3.1 — Observar el nuevo modelo de entrada

Abre `Models/ProductoFormulario.cs` y compáralo con `Models/Producto.cs`. El modelo de dominio posee `Id`, `Nombre` y `Precio`; el de entrada se reduce a los dos campos que el usuario debe rellenar.

```
public sealed class ProductoFormulario
{
    public string? Nombre { get; set; }
    public decimal? Precio { get; set; }
}
```

Predicción previa: ¿qué sucederá con el identificador del producto si alguien añade a mano `Id=99` en la petición de creación? La acción POST utiliza `ProductoFormulario`, que no expone esa propiedad. Por tanto, el binder de ese parámetro no tiene un campo Id en el contrato de entrada que deba completar. Más tarde el controlador construye explícitamente un `Producto` de demostración; no interpreta el valor Id del cliente como identificador definitivo.

Localiza en `ProductosController.cs` la acción GET de creación:

```
[HttpGet]
public IActionResult Crear() => View(new ProductoFormulario());
```

La vista `Crear.cshtml` también declara `@model CursoWeb.Mvc.Models.ProductoFormulario`. Así, el objeto que se utiliza para construir los campos y el que recibe la acción POST comparten contrato. En P2.2 trabajábamos directamente con `Producto`; ahora hemos evitado aceptar desde el formulario una propiedad que corresponde al sistema.

Inspecciona los atributos de entrada que genera la vista. Deben conservar `name="Nombre"` y `name="Precio"`. El tipo C# de destino ha cambiado, pero el navegador continúa enviando los mismos pares de claves y valores. Esto demuestra que **el contrato HTTP y la representación interna de la entidad no necesitan ser la misma clase**.

Estado inicial antes de activar reglas de negocio

Observa que las cuatro anotaciones de validación están comentadas en `ProductoFormulario.cs`. Por ahora una entrada como Nombre = «X», Precio = 7 puede

llegar a la confirmación, porque la longitud mínima no se ha activado. El controlador mantiene una salvaguarda provisional para datos ausentes, de forma que ese estado de aprendizaje no intente desreferenciar un precio nulo; esa salvaguarda todavía no sustituye una validación declarativa adecuada.

Ejecuta una comprobación controlada: con las anotaciones sin activar, envía Nombre = X y Precio = 7. Si recibes una confirmación, has demostrado que **recibir correctamente el valor no equivale a considerarlo válido según una regla que todavía no existe**. Después activaremos StringLength y repetiremos exactamente los mismos campos.

No intentes resolver la diferencia insertando un `if` nuevo en cada vista o sustituyendo Precio por un string. El cambio docente previsto consiste en declarar reglas claras dentro del modelo de entrada.

Paso 2.3.2 — Activar Required, StringLength y Range

Abre de nuevo `Models/ProductoFormulario.cs`. Encontrarás las reglas precedidas por comentarios de una línea `//`, junto a las marcas **PASO 2.3.2**. Quita los prefijos de comentario de **las cuatro anotaciones**, no de las explicaciones que las acompañan. El resultado debe ser:

```
using System.ComponentModel.DataAnnotations;

public sealed class ProductoFormulario
{
    [Required(ErrorMessage = "Indica el nombre.")]
    [StringLength(60, MinimumLength = 2,
        ErrorMessage = "El nombre debe tener entre 2 y 60 caracteres.")]
    public string? Nombre { get; set; }

    [Required(ErrorMessage = "Indica el precio.")]
    [Range(typeof(decimal), "0.01", "9999.99",
        ParseLimitsInInvariantCulture = true,
        ErrorMessage = "El precio debe estar entre 0,01 y 9999,99.")]
    public decimal? Precio { get; set; }
}
```

El atributo `ParseLimitsInInvariantCulture = true` es deliberado: obliga a interpretar los límites 0.01 y 9999.99 según la cultura invariante, aunque el sistema esté configurado en español. Esto **no** cambia automáticamente la cultura con la que el model binder interpreta el campo recibido. El ejemplo omite el espacio de nombres de la clase porque estamos destacando las reglas. En el archivo real **conserva**

`namespace CursoWeb.Mvc.Models;` y la declaración completa. Debes cambiar únicamente las anotaciones indicadas.

Compila de nuevo, ejecuta el formulario y repite la entrada Nombre = X, Precio = 7. Ahora el resultado es diferente: aparece el formulario nuevamente con un error junto a Nombre. En el archivo de modelo puedes encontrar el mismo mensaje que acaba de mostrarse. La acción no ha cambiado su nombre ni su ruta; ha cambiado el criterio utilizado para aceptar la entrada.

Cuatro experimentos en los que cambia una sola variable

Primero envía **Nombre vacío y Precio 7**. Debe aparecer el mensaje «Indica el nombre». Comprueba que el precio se conserva al volver a representar la página. Un error en un campo no debe obligar a escribir otra vez el resto de información que sí era correcta.

Después envía **Nombre X y Precio 7**. Esta vez Nombre no está ausente, pero tiene menos de dos caracteres. Debe mostrarse el mensaje de `StringLength`. Explica por qué no sería apropiado diagnosticar el problema como un fallo del model binder: el binder puede representar perfectamente «X» como string.

En el tercer envío utiliza **Nombre Rotulador y Precio 0**. El precio existe y puede convertirse a decimal, pero está por debajo del mínimo 0,01 definido en `Range`. El mensaje debe aparecer junto al campo Precio. Con **Precio 7** la misma entrada nominal debería ser aceptada.

Por último, comprueba el caso de **Nombre Rotulador y Precio vacío**. El tipo `decimal?` permite expresar un valor ausente antes de aplicar las reglas, y `[Required]` lo considera inválido. Es distinto de un cero fuera de rango. **No confundas tipo anulable y campo de negocio opcional:** hemos elegido una propiedad anulable precisamente para poder representar y diagnosticar la ausencia en la entrada HTTP.

Entrada	Regla que debe intervenir	Resultado del formulario
Nombre vacío, Precio 7	`Required` de Nombre	Se repite Crear con mensaje
Nombre X, Precio 7	`StringLength` de Nombre	Se repite Crear con mensaje
Nombre Rotulador, Precio 0	`Range` de Precio	Se repite Crear con mensaje
Nombre Rotulador, Precio vacío	`Required` de Precio	Se repite Crear con mensaje
Nombre Rotulador, Precio 7	Todas se cumplen	Se muestra Confirmacion

Los primeros cuatro casos son negativos, pero **no son fallos de ejecución del proyecto**. La aplicación debe representarlos de forma comprensible, permitir su corrección y no tratarlos como productos válidos.

Una condición que el navegador puede impedir escribir

Cuando un input tiene `type="number"`, algunos navegadores impiden introducir letras ordinarias en el control. Eso no demuestra que el servidor sea invulnerable a `Precio=abc`. Puedes estudiar la variante usando la pestaña Red y una herramienta de repetición de peticiones que te permita modificar el contenido del POST, siempre manteniendo el token antiforgery real obtenido de una carga actual del formulario.

La entrada `Precio=abc` no se convierte en un decimal. ModelState registra un **error de conversión**, diferente del mensaje de `Range` que corresponde a un número real fuera de los límites. No desactives antiforgery para simplificar la prueba: si el POST no incluye un token válido, recibirás 400 por otro motivo y dejarás de estudiar model binding.

Paso 2.3.3 — Comprender y observar ModelState

En `Controllers/ProductosController.cs` localiza la acción POST. Ahora recibe un `ProductoFormulario entrada` y consulta el estado del enlace y de la validación:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Crear(ProductoFormulario entrada)
{
    if (!ModelState.IsValid)
    {
        return View(entrada);
    }

    // Construir Producto y mostrar Confirmacion...
}
```

Este fragmento es la parte esencial del método, no una sustitución literal de todo el archivo. En el código real hay comentarios, una salvaguarda del estado inicial sin anotaciones y la conversión posterior al tipo de dominio. Conserva esos elementos.

Predicción: si `Nombre="X"`, `ModelState.IsValid` será falso después de activar `StringLength`. La acción ejecutará `return View(entrada)`, sin construir el Producto de confirmación. Si en cambio el nombre es Rotulador y el precio 7, ambas propiedades pueden validarse y el controlador construye el modelo de dominio.

Coloca un punto de interrupción en Visual Studio sobre `if (!ModelState.IsValid)`. Envía primero los datos correctos y después los incorrectos. Observa la propiedad `IsValid` y examina la colección de errores del estado del modelo. No necesitas aprender cada miembro de la API: basta con entender que el estado contiene claves asociadas a los campos y mensajes recogidos durante el procesamiento de la solicitud.

El controlador entrega también un bloque opcional **PASO 2.3.3 INICIO/FIN** que recorre `ModelState` y escribe la clave y el texto de sus errores mediante `Debug.WriteLine`. Si quieres estudiar ese recorrido, quita los delimitadores del comentario C#, compila y envía Nombre = X. Comprueba que el diagnóstico identifica un error asociado a Nombre. Después, desactiva otra vez el bloque si ya no lo necesitas.

No confundas observar con corregir. Un mensaje escrito en la ventana de depuración no modifica la validez de los datos. La decisión de la acción sigue determinada por ModelState. El control de flujo que devuelve la vista es lo que impide que los datos inválidos se traten como aceptados.

¿Por qué aquí no devolvemos automáticamente HTTP 400?

Quien conoce Web API puede recordar el comportamiento de un controlador con `[ApiController]`, donde una entrada inválida puede recibir automáticamente un error HTTP 400. Este formulario MVC devuelve normalmente **200 con el HTML de Crear** cuando la acción hace `return View(entrada)`. El usuario puede ver los mensajes de validación y corregir sus datos.

Eso no significa que el dato inválido haya sido aceptado. La respuesta 200 pertenece a la **representación correcta del formulario con errores**, no a la creación de un producto. Contrástalo con el fallo de antiforgery de P2.1: una petición sin token obtiene 400 antes de ejecutar el procesamiento previsto en la acción. Los dos escenarios tienen objetivos distintos.

Paso 2.3.4 — Mostrar errores sin perder lo que se escribió

Abre `Views/Productos/Crear.cshtml`. Los controles mantienen los Tag Helpers de P2.2, pero ahora el modelo se llama `ProductoFormulario`, y la página contiene elementos para visualizar mensajes:

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
<div>
    <label asp-for="Nombre"></label>
    <input asp-for="Nombre" />
    <span asp-validation-for="Nombre" class="text-danger"></span>
</div>

<div>
    <label asp-for="Precio"></label>
    <input asp-for="Precio" type="number" step="0.01" />
    <span asp-validation-for="Precio" class="text-danger"></span>
</div>
```

Envía **Nombre = Carpeta grande, Precio = -2**. La página debe representar un error asociado al precio, pero no debe borrar «Carpeta grande». En el HTML generado, inspecciona el input Nombre para comprobar que mantiene el valor anterior. El servidor volvió a presentar la vista con `View(entrada)`, y los Tag Helpers pueden utilizar el estado de los valores intentados para reconstruir el formulario.

Ahora cambia temporalmente `asp-validation-summary="ModelOnly"` por `asp-validation-summary="All"`. Repite la misma prueba. Con All, los errores de propiedades pueden mostrarse también dentro del resumen general; con ModelOnly, en cambio, es normal que se vean junto a sus campos sin repetirse arriba. Compara el comportamiento y **restaura ModelOnly** después del experimento.

La validación no depende de que un error sea visualmente rojo. La clase CSS `text-danger` ayuda a distinguir el mensaje, pero la causa es el estado que produce ASP.NET Core cuando comprueba la entrada. Si quitas el span `asp-validation-for`, el error puede seguir existiendo en ModelState aunque el usuario deje de verlo junto al campo. Por eso importan ambas partes: **la regla y su presentación**.

Un mensaje que no es de Range

Envía una cantidad numérica pero fuera de rango, por ejemplo `-5`: debes obtener el mensaje configurado en la anotación Range. Después estudia, si tu herramienta HTTP lo permite, el envío de `Precio=abc` con un token válido. El valor no puede convertirse a decimal; el error es de **model binding**, no de la restricción `Range`.

Anota ambas causas. Cambiar el texto del mensaje de Range no explica un error de conversión. Si el navegador impide introducir letras en el campo numérico, eso no hace innecesaria la validación del servidor, porque otro cliente puede construir directamente una solicitud HTTP.

Paso 2.3.5 — Comparar validación de servidor y validación de navegador

El servidor ya sabe representar los errores. Ahora vamos a añadir, como demostración breve, una mejora de experiencia de usuario: mostrar restricciones en el navegador sin necesitar enviar cada petición inválida.

Abre `Views/Shared/_Layout.cshtml` y observa que contiene `@await RenderSectionAsync("Scripts", required: false)`. Esa sección opcional permite a una página declarar sus propios recursos JavaScript sin obligar a que las demás vistas del catálogo los incluyan.

En `Views/Productos/Crear.cshtml` localiza **PASO 2.3.5 (ACTIVAR)**. El bloque está dentro de un comentario Razor. Quita las marcas de comentario para habilitar lo siguiente:

```
@section Scripts
{
    <partial name="_ValidationScriptsPartial" />
}
```

La parcial `Views/Shared/_ValidationScriptsPartial.cshtml` incluye referencias a tres bibliotecas en orden: **jQuery**, **jQuery Validation** y **jQuery Validation Unobtrusive**. El proyecto usa una CDN, de manera que **necesita conectividad** para completar esta demostración. Un entorno sin Internet debe recibir una copia local preparada de esos recursos; no atribuyas un fallo de descarga a una regla incorrecta del modelo.

Recarga el formulario con F12 → Red abierto y comprueba que los tres scripts se han descargado correctamente. Inspecciona los inputs: sus atributos `data-val` describen reglas derivadas de DataAnnotations. Esos atributos no ejecutan validación por sí solos: necesitan las bibliotecas correspondientes.

Intenta enviar **Nombre = X, Precio = 7**. Si los scripts están cargados y la validación cliente interviene, el navegador puede impedir el POST y mostrar el error sin recorrer el servidor. Comprueba en la pestaña Red si realmente ha existido una nueva petición POST, en vez de deducirlo del aspecto de la pantalla.

Repetir la prueba con JavaScript deshabilitado

Deshabilita temporalmente JavaScript para esa página mediante las herramientas de desarrollo del navegador y recarga el formulario. Repite la entrada inválida. Aunque ya no actúen las bibliotecas cliente, el servidor debe recibir el POST

protegido, evaluar ModelState y volver a mostrar Crear con sus mensajes. **La validación de servidor permanece obligatoria.**

Vuelve a habilitar JavaScript después del experimento para evitar confusiones durante el resto del curso. Si alguna biblioteca no puede descargarse, comprueba la consola del navegador y el estado de cada solicitud de script. El comportamiento del servidor sigue siendo el resultado principal, independientemente de la disponibilidad de esa CDN.

No hagas una regla duplicada y contradictoria en JavaScript. Las DataAnnotations representan la definición central de estas restricciones. El cliente proporciona una experiencia más inmediata cuando puede interpretar sus metadatos, pero no es una frontera de confianza.

Paso 2.3.6 — Aplicación autónoma: modificar una etiqueta sin cambiar el contrato HTTP

Hasta ahora has observado y activado código preparado. En esta actividad debes **escribir una anotación propia**. Abre `Models/ProductoFormulario.cs` y localiza **APLICACION 2.3.6**. Coloca sobre la propiedad `Nombre` el atributo que permite mostrar la etiqueta «Nombre del producto»:

```
[Display(Name = "Nombre del producto")]
```

No cambies el identificador de la propiedad C#. Sigue llamándose `Nombre` y el controlador seguirá recibiéndola con esa clave. Compila, recarga Crear y comprueba que el Label Tag Helper muestra el nuevo texto. Inspecciona el input y verifica que el atributo `name` sigue correspondiendo a `Nombre`.

Este ejercicio exige explicar **dos resultados diferentes**. Visualmente puede cambiar el texto de la etiqueta, porque se ha añadido un metadato de presentación. Funcionalmente no debe cambiar el contrato que utiliza el model binder: el campo de entrada sigue siendo Nombre y las reglas Required y StringLength siguen aplicándose a la misma propiedad.

Envía Rotulador y 7. Si llega a Confirmacion, puedes concluir que el nuevo metadato de presentación no ha roto el flujo POST. Repite Nombre = X para confirmar que continúa funcionando el error de longitud mínima. Si los mensajes aparecen, la propiedad no ha sido renombrada ni la validación se ha eliminado.

La anotación Display no sustituye Required, StringLength o Range. Expresa cómo se presenta un campo, no si el usuario está obligado a rellenarlo.

Reto 2.3 — Una regla adicional introducida desde el controlador

El controlador incluye un bloque comentado **RETO 2.3 INICIO/FIN**. Este ejemplo añade un error específico a ModelState cuando el nombre comienza por `TEST`, ignorando mayúsculas y minúsculas. El objetivo es comprender que la acción puede aportar un error contextual que se presenta con el mismo mecanismo de la vista.

Activa únicamente ese bloque y compila. Envía `Nombre=TESTProducto`, `Precio=7`. El nombre tiene longitud suficiente y el precio cumple el intervalo, pero la acción debe añadir un error y devolver Crear. A continuación, envía `Nombre=Rotulador`, `Precio=7` y verifica que se representa Confirmacion.

El bloque preparado incluye el núcleo:

```
if (entrada.Nombre?.StartsWith("TEST", StringComparison.OrdinalIgnoreCase) == true)
{
    ModelState.AddModelError(nameof(entrada.Nombre),
        "El nombre no puede comenzar por TEST en este ejemplo.");
}
```

No coloques la llamada a AddModelError **después** del `if (!ModelState.IsValid)` si esperas que ese mismo control la detecte. La condición del reto se evalúa antes de tomar la decisión sobre IsValid; el orden es parte de lo que debes razonar.

Este ejemplo demuestra una posibilidad técnica, no una invitación a escribir todas las reglas empresariales dentro de controladores. Para un sistema amplio, la ubicación de esas reglas requiere diseño. En este curso nos interesa comprender que los errores de las anotaciones, los errores de conversión y los añadidos por la acción pueden terminar en la misma presentación Razor.

Probar el resultado conjunto sin falsear la persistencia

Si has realizado la aplicación autónoma de Display y el reto, comprueba finalmente que el catálogo de M01 conserva sus tres productos. Un POST válido todavía devuelve Confirmacion sin guardar, mientras que los POST inválidos se quedan en el formulario. La representación, la protección antiforgery y la validación funcionan conjuntamente sin necesidad de una base de datos.

Diagnóstico de los problemas más habituales

Aparece un 400 antes de que el controlador llegue al bloque de validación. Confirma que la petición contiene un token antiforgery válido y que se dirige a la acción correcta. No deshabilites ValidateAntiForgeryToken para estudiar Required o Range. De lo contrario, habrás cambiado la causa del error que querías observar.

Aparece un 200 pero no hay página de confirmación. Comprueba si el controlador ha devuelto la vista Crear debido a un ModelState inválido. La respuesta 200 puede ser el HTML correcto del formulario con mensajes; no significa que un producto se haya aceptado.

Los mensajes no aparecen junto a los campos. Comprueba las reglas activadas, `ModelState.IsValid` y los `span asp-validation-for` de la vista. Si utilizas `ModelOnly` en el resumen, recuerda que los errores de propiedades pueden no aparecer allí. Prueba temporalmente `All` para estudiar la diferencia.

El precio se borra o no conserva lo escrito. Observa el valor recibido, los atributos que genera Input Tag Helper, la conversión decimal y el contenido de ModelState. Si un formato no puede convertirse, el valor intentado puede conservarse en el estado de enlace; no reemplaces a ciegas el modelo entero por un Producto nuevo al devolver la vista.

La validación funciona con JavaScript, pero deja de hacerlo al desconectarse de Internet. Diferencia los dos mecanismos. La CDN afecta a los scripts de cliente. Una entrada inválida que se envía al servidor con token válido debe seguir siendo detectada mediante ModelState. Investiga por separado la descarga del JavaScript y la ejecución del POST.

Un producto inválido llega a Confirmacion. Comprueba si quedaron comentadas las DataAnnotations del modelo o si algún cambio ha omitido la comprobación de IsValid. Una regla que existe solo como comentario no valida nada. Y un Input Tag Helper no crea de forma automática reglas de negocio que nunca declaramos.

Resultado final de P2.3 y puente hacia P2.4

Al completar el recorrido esencial, el alumno debe poder describir sin ambigüedad las responsabilidades de cada elemento: **el navegador envía campos; el binder los convierte; DataAnnotations expresa restricciones; ModelState reúne**

errores; el controlador decide si devuelve el formulario o la confirmación; y Razor representa valores y mensajes.

El proyecto conserva las reglas activadas y los elementos de validación en el flujo principal. La demostración de scripts y los retos no deben impedir que la jornada avance hacia las operaciones persistentes. **P2.4** incorporará SQL Server Express LocalDB y Entity Framework Core para que crear, editar o eliminar productos produzca cambios reales en el catálogo. Como el alumnado ya conoce EF Core, la atención estará puesta en cómo se combina el modelo de entrada validado con las acciones MVC y las vistas Razor.

Cuaderno de profundización opcional P2.3 — Cuatro causas de fallo diferentes

Conserva el estado de `PROYECTO/2.3/CursoWeb.Mvc` después de activar las anotaciones del paso 2.3.2. El objetivo es diferenciar **ausencia**, **error de conversión**, **regla de negocio** y **ausencia de token**. Las cuatro situaciones pueden impedir un resultado positivo, pero requieren razonamientos distintos. No debes quitar validaciones para que una entrada incorrecta parezca válida.

Experimento A: nombre ausente. Desde `/Productos/Crear` envía el campo Nombre vacío y un precio correcto. Si la validación cliente impide enviar, utiliza el procedimiento de 2.3.5 para observar también la respuesta del servidor. Anota el mensaje asociado con `Required` y comprueba que se muestra en la zona de error del campo. El dato ha llegado vacío o no ha proporcionado un valor aceptable, pero el problema no está en SQL Server: P2.3 todavía no inserta productos.

Experimento B: nombre presente pero demasiado corto. Envía «X» con precio 7 y observa el mensaje de `StringLength`. Contrasta con una entrada «Ana». Las dos solicitudes tienen un valor de `Nombre` convertible a `string`; lo que cambia es una condición sobre su longitud. La respuesta positiva del segundo caso no implica que se haya ejecutado un INSERT. La utilidad de este contraste es comprobar que el binder y la validación de negocio resuelven preguntas diferentes.

Experimento C: precio no convertible. Utiliza la petición de servidor prevista en el punto 2.3 para enviar una cadena alfabética como precio. El navegador podría impedir escribirla en un campo `type="number"`, así que no tomes la restricción de interfaz como prueba suficiente. Cuando el texto no pueda convertirse a `decimal?`, debería existir un error de `ModelState`. Comprueba qué valor del formulario se

conserva al volver a presentar la vista y justifica por qué una excepción de conversión no se corrige ampliando el intervalo de `Range`.

Experimento D: método correcto, protección ausente. Fabrica un POST con campos válidos pero sin token antiforgery, tal como hiciste en P2.1. El servidor debe rechazarlo antes de considerar la entrada como un formulario válido. No es correcto interpretar ese 400 como el mensaje de `Required` o `Range`. La protección de procedencia y la validación de datos son filtros independientes; una aplicación segura necesita ambos cuando corresponden.

Interpretación conjunta. Elabora una tabla propia con cuatro columnas: valor enviado, etapa que detecta el problema, respuesta observable y archivo responsable. Para A y B mirarás el contrato de `ProductoFormulario` y la representación de errores; para C examinarás binder y conversión; para D revisarás el endpoint y su antiforgery. El objetivo no es contar cuántos mensajes existen, sino decidir qué comprobar primero cuando un usuario comunica que «el formulario no funciona».

Solución razonada del análisis. A y B pertenecen a validación declarativa, con mensajes distintos porque expresan requisitos distintos; C falla antes de disponer de un precio numérico correcto; D no supera la comprobación de seguridad del POST. En los tres primeros casos, el controlador puede devolver la misma vista y conservar valores; en el cuarto, la solicitud puede rechazarse antes de ejecutar la acción. Esta diferencia también explica por qué un HTTP 200 con errores de formulario y un HTTP 400 no son resultados contradictorios.

Consolidación: después de todas las pruebas, vuelve a `/Productos` y comprueba que sigue apareciendo el catálogo fijo. Habremos demostrado la separación entre recibir, convertir y validar, pero aún no la escritura persistente. La teoría 2.3 describe causalmente este mismo recorrido en su apartado de profundización aplicada.

Resolución guiada P2.3 — Una investigación completa de los mensajes de validación

Esta práctica de transferencia comienza desde el estado validado de `PROYECTO/2.3/CursoWeb.Mvc`, después de activar `Required`, `StringLength` y `Range`, y después de estudiar `ModelState`. No cambia la definición del formulario ni introduce clases adicionales: utiliza deliberadamente la implementación ya preparada para que puedas concentrarte en comprender la causa de cada resultado.

Una operación aparentemente idéntica produce dos mensajes distintos

Dos personas comunican que «el precio no se acepta». La primera ha dejado el campo vacío; la segunda ha escrito cero. Aunque ambas pueden ver el formulario de nuevo, no han enviado exactamente la misma información. En el primer caso falta un valor significativo para una propiedad que se ha declarado obligatoria; en el segundo existe un valor numérico que no supera el mínimo de negocio. Abre `Models/ProductoFormulario.cs` y busca las reglas correspondientes a `Precio`. Señala qué anotación expresa la obligatoriedad y cuál expresa el intervalo aceptable.

Prueba ambos valores sin modificar las reglas. Si la validación cliente bloquea el envío, observa el modo de deshabilitar JavaScript ya explicado en 2.3.5 para provocar la comprobación de servidor. La respuesta esperada es una vista que conserva los datos del usuario y presenta el mensaje correspondiente. No concluyas «no funciona el formulario» por el mero hecho de recibir otra vez HTML: en este punto, la devolución de la vista con errores es precisamente el contrato correcto.

El texto no convertible es distinto del número fuera de rango

En el laboratorio se utilizaron valores enteros sencillos para no introducir diferencias de cultura. Ahora considera un cliente que remite literalmente `Precio=abc`. El dato enviado no se puede interpretar como `decimal?`, por lo que el proceso de enlace de modelos debe comunicar un problema antes de aplicar con normalidad la regla de intervalo sobre un número válido. Esta observación importa cuando un usuario comunica un mensaje distinto del que esperamos de `Range`.

Compara la petición HTTP que lleva `Precio=abc` con otra que lleva `Precio=0`. Ambas deben rechazarse, pero no por la misma razón. Para la primera revisamos el binder y el tipo de la propiedad; para la segunda revisamos el requisito de negocio. Si una persona propone aumentar el máximo del `Range` para permitir letras, explica por qué no puede resolver una conversión de tipos. Si otra propone sustituir `decimal?` por `string` y eliminar validaciones, explica qué contrato estaríamos debilitando.

El formulario conserva lo escrito incluso después de un error

El paso 2.3.4 explica que, cuando la entrada es inválida, la acción devuelve la vista con el objeto de entrada. Es una decisión pedagógica importante: queremos que el usuario pueda corregir solo lo incorrecto, no volver a escribir todos los campos. Introduce un nombre aceptable y un precio inválido, observa el error de precio y comprueba que el nombre sigue en el campo. Después introduce un nombre inválido y un precio aceptable; comprueba el comportamiento inverso.

No confundas «valor conservado para corregirlo» con «valor guardado». El primero está relacionado con el estado de la solicitud y el modelo que vuelve a la vista. El segundo exigiría una operación persistente que todavía no existe en P2.3. Recuerda que en la fase siguiente del módulo utilizaremos esta misma experiencia para asegurarnos de que la base de datos no cambia cuando `ModelState.IsValid` es falso.

Un error de validación que no se ve no es necesariamente un error que no existe

La vista utiliza elementos de visualización para errores del modelo y de campos concretos. Examina la posición de `asp-validation-summary` y los `span` con `asp-validation-for`. Si el error pertenece a `Nombre`, la zona asociada a esa propiedad es su representación natural. Un resumen configurado como `ModelOnly` no tiene por qué mostrar todos los mensajes de propiedades. Esta distinción ayuda a interpretar capturas de pantalla de una aplicación: no es suficiente que el HTML contenga un resumen vacío para concluir que el servidor ha aceptado los datos.

Verifica también que el formulario contiene los atributos generados para los campos. Esa conexión permite que los scripts de validación cliente ofrezcan una respuesta inmediata, pero la comprobación en servidor sigue siendo necesaria. Un navegador puede tener JavaScript desactivado, una petición puede enviarse desde una herramienta de pruebas o los datos pueden manipularse antes del POST. No se debe organizar la persistencia sobre la suposición de que todo cliente es honesto.

Cómo redactar un diagnóstico útil para otra persona

Prepara cuatro notas, una por causa: ausencia de nombre, nombre de longitud incorrecta, precio no convertible y POST sin token. Cada nota debe incluir el valor concreto que se envió, el resultado observado y el archivo donde investigarías el comportamiento. En la primera y la segunda citarás DataAnnotations; en la tercera hablarás del binder; en la cuarta de la protección antiforgery. La respuesta debe permitir a otro alumno reproducir la situación sin que tú estés delante.

Finalmente envía una entrada completamente válida y comprueba el mensaje positivo. Después abre el listado: los datos de la confirmación no deben haberse añadido al catálogo. Esta última verificación prepara el paso siguiente: en P2.4, el mismo control de `ModelState` protegerá un INSERT real. La continuidad no está en repetir el mismo mensaje en dos documentos, sino en comprobar qué efecto nuevo se añade al aceptar un formulario válido.

P2.4. Crear, listar, editar y eliminar productos persistentes

Situación inicial, objetivos y preparación del laboratorio

Durante P2.1 el alumno observó y activó un formulario HTML con POST; en P2.2 lo transformó mediante Tag Helpers; en P2.3 añadió reglas de validación para distinguir un producto aceptable de una entrada incorrecta. Hasta aquí, la confirmación solo mostraba los datos recibidos. La nueva práctica completa el recorrido: **guardar el producto, encontrarlo en el catálogo, modificarlo y eliminarlo**, conservando las reglas de validación y la protección antiforgery.

P2.4 es la práctica central del segundo día. El objetivo esencial es comprender cómo un formulario MVC produce cambios verificables en SQL Server Express LocalDB. El alumnado ya conoce C# y EF Core, por lo que se utilizarán directamente `DbContext`, consultas LINQ, `Add`, `Remove` y `SaveChangesAsync`, sin convertir la jornada en otro curso de persistencia.

Teoría relacionada: capítulo 2.4 del manual del módulo. Lee especialmente la diferencia entre `Producto` y `ProductoFormulario`, `EnsureCreated` y migraciones, consultas sin seguimiento, Post/Redirect/Get y la necesidad de editar una entidad existente sin aceptar todas sus propiedades desde el navegador.

Proyecto a abrir: `M02_FORMULARIOS_RAZOR/PROYECTO/2.4/CursoWeb.Mvc/CursoWeb.Mvc.csproj`. Es una instantánea específica del catálogo, basada en el recorrido de los puntos anteriores. **No modifiques los proyectos de M01 ni de P2.3.** Los controles de entrada conservan las DataAnnotations que activaste conceptualmente en la práctica anterior, mientras que las operaciones persistentes se incorporarán por etapas.

Requisitos específicos: Windows, Visual Studio con carga de ASP.NET y .NET SDK 8, SQL Server Express LocalDB instalado y una cuenta con permisos sobre su instancia local. No necesitas un servidor SQL remoto, contenedores ni infraestructura de identidades. Es preferible que el proyecto se ejecute con un solo proceso Kestrel para que las comprobaciones reflejen siempre la misma aplicación.

Para comprobar que Windows reconoce LocalDB, abre una terminal y ejecuta:

```
sqllocaldb info
```

En una instalación convencional debería aparecer la instancia `MSSQLLocalDB`. Si la instalación no existe, hay que preparar ese componente del laboratorio antes de realizar el CRUD. La ausencia de LocalDB **no es un problema de Razor ni de routing**, por lo que no debemos resolverla cambiando las vistas.

Mapa completo de la práctica

Paso	Archivo concreto	Acción principal	Evidencia
2.4.1	`Program.cs`, `Data/AppDbContext.cs`, `appsettings.json`	**OBSERVAR** conexión, esquema y semilla inicial	Arranque con base y tres productos
2.4.2	`Controllers/ProductosController.cs`, `Views/Productos/Index.cshtml`, `Detalle.cshtml`	**OBSERVAR** consultas de lectura desde LocalDB	Lista y ficha desde SQL, 404 para Id inexistente
2.4.3	`Controllers/ProductosController.cs`, `Views/Productos/Crear.cshtml`	**ACTIVAR** el POST protegido de creación	302 a nueva ficha, producto guardado
2.4.4	`ProductosController.cs`, `Index.cshtml`, `Editar.cshtml`	**ACTIVAR** GET y POST de edición y su enlace	Nombre/precio modificados en el mismo Id
2.4.5	`ProductosController.cs`, `Index.cshtml`, `Eliminar.cshtml`	**ACTIVAR** GET y POST de eliminación y su enlace	GET confirma, POST elimina, detalle pasa a 404
2.4.6	`Program.cs`, todas las acciones de CRUD	**COMPROBAR** persistencia tras reinicio	Cambios permanecen, semilla no se duplica
Aplicación 2.4.7	`Views/Productos/Index.cshtml`	**ESCRIBIR** aviso para catálogo vacío	Aparece solo si Model.Count == 0
Reto 2.4	`ProductosController.cs`, `Index.cshtml`	**ACTIVAR** búsqueda GET por nombre	Listado filtrado sin modificar la base

Los pasos **2.4.1–2.4.6** constituyen el recorrido esencial de la clase. La aplicación autónoma y el reto son variantes de consolidación, no condiciones para avanzar hacia P2.5. El alumnado debe poder explicar por qué cada operación es una lectura o escritura y qué solicitud HTTP la desencadena.

Paso 2.4.1 — Crear el contexto y reconocer la base del laboratorio

Desde la raíz del repositorio, entra en el proyecto y compílalo:

```
cd M02_FORMULARIOS_RAZOR/PROYECTO/2.4/CursoWeb.Mvc
dotnet restore
```

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

El primer arranque **se conecta a LocalDB antes de escuchar peticiones**. Si la instancia no está disponible, puede aparecer una excepción durante la inicialización y Kestrel no llegará a escuchar. En ese caso, revisa primero `sqllocaldb info`, la cadena de conexión y los mensajes concretos de la consola. No alteres el controlador del catálogo para disimular un fallo de base de datos.

Abre `appsettings.json` y localiza `ConnectionStrings:Productos`. La conexión de ejemplo utiliza `Server=(localdb)\\MSSQLLocalDB` y una base llamada `CursoWebProductos`. La autenticación es integrada. El nombre de la base es parte del escenario local; si necesitas aislarlo de otro ejercicio existente, usa un nombre nuevo. **No borres ni reemplaces una base que puedas haber creado para otro proyecto**.

En `Data/AppDbContext.cs` identifica las piezas esenciales del contexto:

```
public sealed class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    public DbSet<Producto> Productos => Set<Producto>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Producto>()
            .Property(p => p.Precio).HasPrecision(10, 2);
    }
}
```

La propiedad `Productos` representa el conjunto consultable de entidades. La configuración de `Precio` fija la precisión decimal utilizada por el proveedor SQL Server. No hay que reimplementar un almacén en memoria para «facilitar» la práctica; queremos estudiar realmente un dato persistente.

Localiza a continuación en `Program.cs` el **PASO 2.4.1**: lectura de la cadena de conexión, registro del contexto con `AddDbContext`, creación del ámbito y llamada `EnsureCreated()`.

Pregunta previa: si `EnsureCreated` crea la base, ¿añadirá también tres productos cada vez que ejecutes la aplicación? Observa que el programa solo ejecuta `AddRange` cuando `baseCreada` es verdadero. Ese resultado corresponde al primer arranque sobre una base nueva, no a los siguientes.

La semilla inicial y su límite

Cuando la base acaba de crearse, el programa inserta Cuaderno, Lápiz y Carpeta. Esta semilla permite demostrar que un catálogo que en M01 procedía de objetos fijos ahora se obtiene de SQL Server. El mecanismo es deliberadamente simple y **no constituye una estrategia general de migraciones**.

`EnsureCreated` y las migraciones no son intercambiables. La primera opción resulta apropiada para una base nueva de demostración, pero no actualiza automáticamente una base existente cuando modificamos el esquema. No combines después `Migrate()` sobre esta misma base creyendo que estás «completando» el ejemplo.

Comprobación: cuando Kestrel comience a escuchar en el puerto 5091, abre `http://localhost:5091/Productos`. Deberías ver tres elementos con los nombres y precios esperados. Detén y arranca de nuevo el proyecto. Comprueba que siguen siendo tres y que no se han duplicado.

Aislar una base propia sin modificar el repositorio

Si ya existe una base denominada CursoWebProductos que no quieres compartir, utiliza una base de laboratorio distinta suministrando una cadena en una variable de entorno de PowerShell **solo para tu sesión**:

```
$env:ConnectionStrings__Productos = 'Server=(localdb)
\MSSQLLocalDB;Database=CursoWebProductosAlumno;Trusted_Connection=True;TrustServerCer
tificate=True'
dotnet run --no-launch-profile --urls http://localhost:5091
```

La doble barra baja en `ConnectionStrings__Productos` corresponde a la jerarquía de configuración de .NET; permite sustituir la cadena del archivo sin cambiar el código. Si vuelves a ejecutar el proyecto sin esa variable, consultarás la base configurada en appsettings.json. Anota conscientemente qué conexión estás utilizando: **comparar bases diferentes puede hacerte pensar que una inserción ha desaparecido**.

Paso 2.4.2 — Leer productos desde SQL y observar su ficha

En `ProductosController.cs` encuentra el método `Index` marcado como **PASO 2.4.2**. Conserva la acción GET que ya conoces, pero su lista procede de EF Core:

```
[HttpGet]
public async Task<IActionResult> Index()
{
    var productos = await _db.Productos
        .AsNoTracking()
        .OrderBy(p => p.Id)
        .ToListAsync();

    return View(productos);
}
```

Explica por qué no hay `SaveChangesAsync` en este método. Es una consulta de lectura. `AsNoTracking` evita el seguimiento de objetos que solamente van a mostrarse. `OrderBy` hace explícito un orden de resultados. La vista `Index.cshtml` recibe una colección como en M01, de modo que no hace falta reconstruir su layout, sus tarjetas parciales ni su estructura de navegación.

Visita `/Productos/Detalle/2` y comprueba que devuelve la ficha de Lápiz si esa entidad conserva el identificador 2 en una base recién sembrada. **No presupongas ese número si utilizas una base que ya contenía registros**: en ese caso, selecciona un enlace de detalle del listado para obtener un Id real.

Ahora solicita un identificador positivo inexistente, por ejemplo `/Productos/Detalle/999999`. El resultado esperado es **404 Not Found** si no existe una fila con ese número. Solicita también `/Productos/Detalle/0`: el controlador responde con un error de solicitud por identificador no válido. Distingue ambos casos. Uno representa una entidad ausente; el otro representa un parámetro que la acción rechaza.

Predicción: ¿si reiniciamos Kestrel cambiará por sí solo el resultado de `Index`? No. La lectura consulta la misma base local y debe recuperar los datos que permanecen en ella.

Paso 2.4.3 — Activar Create y comprobar un INSERT real

Hasta aquí hemos leído datos persistentes. El siguiente paso requiere introducir un cambio de estado. Abre `/Productos/Crear`; verás un formulario con `Nombre`, `Precio` y el botón **Guardar producto**. La vista utiliza el mismo `ProductoFormulario` validado en P2.3, con Form, Label e Input Tag Helpers.

Antes de modificar nada, prueba a enviar un producto, por ejemplo **Nombre = Rotulador, Precio = 7**. En el estado inicial de 2.4 la acción POST de creación permanece comentada. No debería producirse una inserción; normalmente el

servidor devolverá **405 Method Not Allowed** al no existir la acción POST correspondiente. La página del formulario GET sí está disponible. Comprueba que Rotulador no aparece en `/Productos`.

Abre `Controllers/ProductosController.cs`, localiza **PASO 2.4.3 INICIO** y **PASO 2.4.3 FIN**, y retira solamente los delimitadores `/*` y `*/` del bloque completo. Conserva las condiciones, atributos y comentarios del método. El núcleo activo debe contener:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Crear(ProductoFormulario entrada)
{
    if (!ModelState.IsValid)
        return View(entrada);

    var producto = new Producto
    {
        Nombre = entrada.Nombre!,
        Precio = entrada.Precio!.Value
    };

    _db.Productos.Add(producto);
    await _db.SaveChangesAsync();
    return RedirectToAction(nameof(Detalle), new { id = producto.Id });
}
```

Compila, inicia el proyecto y vuelve a enviar Rotulador, 7 **desde el formulario real**. El resultado esperado ya no es una confirmación temporal de P2.3, sino una redirección hacia la ficha del producto creado. Inspecciona F12 → Red: deberías observar el POST, seguido por un GET al detalle del nuevo identificador.

No presupongas que el Id nuevo es siempre 4. Si la base ya contenía productos, EF Core y SQL Server pueden asignar otro valor. Utiliza la dirección de detalle producida por el servidor para identificar la fila recién guardada. Ahora visita `/Productos`: debe aparecer el nuevo producto junto a los existentes.

Explicar qué línea hace cada trabajo

La comprobación `ModelState.IsValid` protege el paso hacia la persistencia. El objeto de entrada no contiene Id; el controlador crea explícitamente una entidad `Producto`. `Add` indica al contexto que la entidad debe insertarse. `SaveChangesAsync` ejecuta el cambio en SQL Server. Finalmente, `RedirectToAction` utiliza el Id que ya existe para mostrar una página GET.

Si el alumno elimina `SaveChangesAsync`, el hecho de haber ejecutado `Add` en el contexto no bastará para garantizar que exista una fila persistente. Si elimina la condición de validación, una petición inválida podría avanzar hacia la escritura. Si devuelve simplemente la vista `Detalle` después de POST sin redirección, estaría cambiando el patrón de navegación que queremos observar.

Una creación inválida no puede convertirse en un registro

Envía ahora **Nombre = X, Precio = 7**. La anotación StringLength rechaza el nombre demasiado corto, y la acción debe devolver otra vez la vista Crear con su mensaje. No debe aparecer un nuevo producto de nombre X en el catálogo. Envía después **Nombre = Rotulador inválido, Precio = 0**: la regla de precio rechaza ese importe y tampoco debe producirse una inserción.

En estos ejemplos puede mostrarse una respuesta HTML con estado 200, porque el controlador devuelve de forma válida **la página del formulario con errores**. No se trata de una creación exitosa ni exige una redirección. La evidencia determinante es que `SaveChangesAsync` no se ejecuta y el listado sigue sin contener un registro correspondiente a esos intentos.

Como contraste, haz un POST directo sin token antiforgery válido, manteniendo la acción protegida. Debería rechazarse con **400**. No desactives antiforgery para «facilitar» las pruebas: la inserción en una base real hace aún más importante conservar la protección que ya aprendimos.

Paso 2.4.4 — Activar Edit y modificar el mismo identificador

La edición introduce una segunda operación persistente: ya no queremos insertar otra entidad, sino localizar **una fila que existe** y modificar sus dos campos editables.

En `ProductosController.cs` localiza el bloque **PASO 2.4.4 INICIO/FIN** y descomenta los métodos GET y POST `Editar` al mismo tiempo. Después abre `Views/Productos/Index.cshtml` y activa el enlace `Editar` señalado como **PASO 2.4.4**. No actives todavía el enlace `Eliminar`.

La acción GET carga desde LocalDB el producto seleccionado y crea un `ProductoFormulario` cuyos campos ya contienen Nombre y Precio. La acción POST recibe el modelo de entrada validado, recupera la entidad existente y asigna solamente las propiedades permitidas:

```
producto.Nombre = entrada.Nombre!;
producto.Precio = entrada.Precio!.Value;
await _db.SaveChangesAsync();

return RedirectToAction(nameof(Detalle), new { id });
```

El controlador no debe sustituir la identidad del producto con un valor `Id` enviado desde un formulario arbitrario. La URL contiene el identificador de la entidad que queremos editar, y el método verifica que esa fila existe. Es una continuación del contrato de entrada separado que aprendiste en P2.3.

Prueba con un registro creado por ti en el paso anterior, no con un producto de un proyecto ajeno. Abre su enlace Editar y cambia el nombre a **Rotulador azul** y el precio a **8**. Guarda. Comprueba que el navegador llega a la misma ficha de identificador y que los nuevos valores aparecen tanto allí como en `/Productos`.

No tiene que existir un segundo registro «Rotulador azul» con otro Id: estamos modificando la fila existente. Si aparecen dos, revisa que hayas activado la acción Editar y que no estés enviando el formulario a Crear.

La edición también puede fallar por un dato inválido

Vuelve a Editar ese mismo producto e introduce un nombre de un solo carácter. Debe mostrarse el formulario con error y conservarse el Id de la URL para que puedas corregirlo. En el código del controlador observa la línea:

```
ViewData["ProductoId"] = id;
return View(entrada);
```

Ésta es una diferencia respecto del POST de creación. La vista `Editar.cshtml` necesita un Id para generar su propia acción de envío, porque la modificación opera sobre un registro existente. Si el controlador devolviera la vista inválida sin restaurar el Id, el formulario podría generar una URL incorrecta para un segundo intento.

Después corrige el nombre a Rotulador azul y el precio a 8. La segunda petición válida debe actualizar la **misma** entidad. Para confirmar el resultado, vuelve a la ficha mediante el enlace del catálogo; no te limites a leer un mensaje de éxito sin comprobar el valor persistido.

Cuando la fila no existe

Prueba una dirección de edición con un Id positivo que no exista en la base. La acción GET debe devolver **404**. Lo mismo ocurre si la fila desaparece antes de procesar un POST de edición: el controlador comprueba la existencia al recibir el envío. No debemos transformar una edición de un registro ausente en una creación automática.

Este ejercicio no implementa control de concurrencia entre varios usuarios. Su objetivo es comprender el recorrido GET → formulario prellenado → POST validado → entidad seguida → SaveChangesAsync → GET de detalle. No presentaremos ese comportamiento como solución universal para conflictos de escritura simultánea.

Paso 2.4.5 — Activar Delete: confirmar con GET y borrar con POST

El recorrido de eliminación debe distinguir expresamente **abrir la página de confirmación** de **ejecutar un borrado**. En el controlador encontrarás los métodos de eliminación dentro de las marcas **PASO 2.4.5 INICIO/FIN**. Descomenta el bloque y activa a continuación el enlace `Eliminar` de `Views/Productos/Index.cshtml`.

Desde el catálogo abre Eliminar para el **producto creado durante esta práctica**. La página `Eliminar.cshtml` muestra su nombre e identificador y ofrece un botón «Sí, eliminar». Todavía no pulses el botón. Regresa al catálogo y comprueba que el producto sigue presente: **la solicitud GET no debe modificar LocalDB**.

La vista tiene un formulario generado mediante Form Tag Helper, con `method="post"` y `asp-route-id` para el producto seleccionado. El botón origina un POST a la acción lógica `Eliminar`. En el controlador el método C# se llama `EliminarConfirmado`, pero el atributo `[ActionName("Eliminar")]` lo expone con ese nombre de acción:

```
[HttpPost, ActionName("Eliminar")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> EliminarConfirmado(int id)
{
    if (id <= 0) return BadRequest();
    var producto = await _db.Productos
        .FirstOrDefaultAsync(p => p.Id == id);
    if (producto is null) return NotFound();

    _db.Productos.Remove(producto);
    await _db.SaveChangesAsync();
```

```
    return RedirectToAction(nameof(Index));
}
```

La acción exige token antiforgery, comprueba la existencia del producto, ejecuta la eliminación mediante EF Core y redirige al listado. No implementes la operación con un enlace GET que borre directamente el registro: eso convertiría una navegación ordinaria en un cambio destructivo.

Prueba completa: anota el Id del producto propio. Abre Eliminar, comprueba que sigue existiendo antes de confirmar y pulsa «Sí, eliminar». Debes regresar al catálogo y ya no ver ese producto. Si solicitas el detalle de su antiguo Id, el resultado esperado es **404 Not Found**. La ausencia del nombre en un único elemento visual es una evidencia más débil que comprobar también la URL de detalle.

Un POST de borrado sin token no debe funcionar

Desde una terminal PowerShell puedes enviar deliberadamente un POST sin token a la ruta de un producto de prueba:

```
curl.exe -i -X POST http://localhost:5091/Productos/Eliminar/999999
```

Con el método POST activo y protegido, el servidor debe rechazar la solicitud normalmente con **400**, antes de ejecutar la operación. Utilizamos un Id inexistente para que la comprobación no comprometa ningún producto de laboratorio, pero no debemos confiar solamente en eso: la protección relevante es el token. No repitas pruebas destructivas sobre identificadores de registros ajenos.

Al terminar el paso, los tres productos iniciales deberían continuar disponibles si solo has eliminado el producto creado para el ejercicio. No necesitas borrar la semilla para demostrar el funcionamiento de Delete.

Paso 2.4.6 — Verificar persistencia tras reiniciar Kestrel

Una aplicación puede representar una página correcta aunque sus datos vivan solamente en memoria. Por eso la validación final de P2.4 debe incluir **detener y reiniciar el servidor**.

Antes del reinicio, crea un producto adicional de laboratorio: **Nombre = Bloc de dibujo, Precio = 12**. Comprueba que su ficha tiene un Id propio. Edita su precio a

14 y visita el detalle otra vez. Debe mostrar ese precio modificado. Si has eliminado previamente Rotulador azul, comprueba también que su antiguo Id devuelve 404.

Detén Kestrel de forma normal. Vuelve a arrancar **el mismo proyecto con la misma cadena de conexión**. Abre el listado, la ficha de Bloc de dibujo y la antigua dirección del producto eliminado. El nuevo producto debe conservarse con precio 14, y el eliminado debe continuar ausente.

No cambies de base de datos entre las dos ejecuciones. Si utilizaste `ConnectionStrings__Productos` para trabajar con una base aislada, debes mantenerla al reiniciar. De lo contrario puedes observar un catálogo diferente y creer erróneamente que se han perdido o restaurado datos.

Comprueba además que los productos iniciales no aparecen duplicados. El bloque de `EnsureCreated()` incluye una condición para sembrarlos únicamente cuando la base acaba de ser creada, no cada vez que ejecutamos el proyecto. La permanencia de cambios y la ausencia de reinserciones son dos caras del mismo comportamiento persistente.

Qué demuestra y qué no demuestra esta secuencia

La secuencia demuestra que el estado de la base de datos **no depende de que el proceso Kestrel permanezca vivo**. No demuestra, sin embargo, que el CRUD tenga permisos empresariales, auditoría de operaciones, control de concurrencia, resistencia frente a caídas de red o una estrategia de evolución de esquemas. Es un laboratorio MVC con LocalDB diseñado para identificar la responsabilidad de cada componente.

En la práctica con seis horas, podemos efectuar un recorrido compacto: crear un registro, editarlo, verificar el nuevo estado, eliminarlo, crear otro y comprobar su persistencia tras el reinicio. Las actividades siguientes sirven para ampliar el aprendizaje sin impedir que el aula avance hacia Razor Pages.

Aplicación autónoma 2.4.7 — Representar una colección vacía

Una lista de productos puede quedar vacía de manera legítima. No debemos confundir ese estado con la ausencia del modelo `null` ni mostrar siempre un mensaje de «error de catálogo».

Abre `Views/Productos/Index.cshtml` y encuentra **APLICACION 2.4.7 (ESCRIBIR)**. Allí hay una indicación, pero no una solución activada. Debes escribir un bloque Razor

que, cuando `Model` sea una colección válida y `Model.Count == 0`, represente exactamente el elemento:

```
<p id="catalogo-vacio">No hay productos guardados.</p>
```

No debe aparecer cuando existen uno o más productos. Sitúa la condición dentro de la rama que ya ha comprobado que `Model` no es `null`; así evitas tratar como equivalentes dos estados que tienen causas diferentes.

Para probarlo de forma segura es preferible una **base de laboratorio aislada**. Si necesitas dejar la colección vacía para estudiar el resultado, utiliza solamente datos de prueba propios y no elimines de manera indiscriminada un catálogo que quieras conservar. Puedes razonar primero el resultado con una colección vacía sin efectuar ningún borrado: la finalidad es dominar la vista, no destruir el escenario compartido.

Solución de referencia tras intentar el ejercicio

```
@if (Model.Count == 0)
{
    <p id="catalogo-vacio">No hay productos guardados.</p>
}
```

Este fragmento presupone que está dentro del bloque `else` que ya garantiza que `Model` no es nulo. Su posición es parte de la solución: si lo colocas fuera de una comprobación de nulabilidad, deberías adaptar la condición para que no se evalúe Count sobre un objeto nulo.

Reto 2.4 — Buscar productos sin modificar la base

El reto consiste en reutilizar lo aprendido de routing, model binding y consultas de EF Core para implementar una **búsqueda GET**. El objetivo es filtrar productos por una parte de su nombre y mostrar el resultado mediante la misma vista Index.

En `ProductosController.cs` encontrarás el bloque **RETO 2.4 INICIO/FIN**, inicialmente comentado. Su acción recibe un parámetro simple `string? nombre` desde la query string:

```
[HttpGet("/catalogo/buscar")]
public async Task<IActionResult> Buscar(string? nombre)
{
    var consulta = _db.Productos.AsNoTracking();

    if (!string.IsNullOrWhiteSpace(nombre))
```

```
        consulta = consulta.Where(p => p.Nombre.Contains(nombre));

    var encontrados = await consulta.OrderBy(p => p.Id).ToListAsync();
    return View("Index", encontrados);
}
```

Después de activarlo, abre `Views/Productos/Index.cshtml` y localiza el formulario GET comentado como **RETO 2.4**. Actívalo también. La etiqueta del campo permite escribir un nombre, y el formulario envía `nombre` como parámetro de consulta, no como un nuevo producto.

Prueba `/catalogo/buscar?nombre=Car` después de confirmar que Carpeta existe en tu base. El catálogo filtrado debería mostrar los productos cuyo nombre cumpla el criterio según la intercalación configurada en SQL Server. Prueba también una cadena que no aparezca en ningún nombre y comprueba que obtienes una colección vacía, no un error de base de datos.

Este ejemplo usa `Contains` dentro de una consulta EF Core traducible por el proveedor SQL Server. La sensibilidad a mayúsculas, minúsculas y acentos puede depender de la **collation** de la base: no asumas que se comporta de forma idéntica en cualquier instalación. La actividad no implementa búsqueda de texto completo ni presupone índices especializados.

Comprobación de que no hay escritura: ejecuta la búsqueda, vuelve al catálogo y verifica que los registros siguen iguales. No debe existir un `SaveChangesAsync` en la acción Buscar. Una consulta GET no tiene por qué convertir el resultado en una entidad nueva ni modificar el precio de ningún producto.

Cuaderno de diagnóstico de P2.4

El objetivo del diagnóstico es decidir **qué archivo y qué componente** explican un resultado, sin modificar simultáneamente controlador, base de datos y vista.

La aplicación ni siquiera inicia. Si aparece una excepción en `EnsureCreated`, comprueba LocalDB y la cadena de conexión antes de mirar el routing. `Program.cs` ejecuta la inicialización antes de escuchar peticiones.

El catálogo muestra cero elementos en una base que ya existía. Averigua si estás utilizando la base esperada. La semilla se aplica al crear la base por primera vez; no es una operación de restauración automática. No borres bases de terceros para forzar que vuelvan a aparecer los tres datos de demostración.

El formulario Crear abre, pero Enviar devuelve 405. Comprueba si has activado el bloque POST de 2.4.3. En el estado inicial del proyecto se entrega comentado para permitir observar esa diferencia. El GET puede funcionar perfectamente aunque todavía no exista POST.

La creación muestra mensajes pero no aparece el producto. Investiga `ModelState.IsValid`: una entrada inválida debe devolver Crear sin llamar a `SaveChangesAsync`. El estado 200 de una vista de error no significa que se haya persistido un registro.

El POST sin token devuelve 400. Es el comportamiento previsto cuando está activo `ValidateAntiForgeryToken`. Si el formulario normal también obtiene 400, inspecciona el token generado por Form Tag Helper y las cookies del navegador antes de retirar protecciones.

Editar abre pero al corregir un dato inválido cambia de destino. Revisa si el POST repone `ViewData["ProductoId"]` antes de devolver `View(entrada)`. La vista necesita ese valor para construir la ruta de la siguiente tentativa.

Eliminar desaparece de la pantalla, pero el detalle antiguo sigue respondiendo 200. Examina si realmente se ejecutó el POST de eliminación, si se llamó a `SaveChangesAsync` y si el proceso utiliza la misma base. Ocultar un enlace en la vista no elimina una fila.

Después de reiniciar aparecen productos diferentes. Comprueba el nombre de base, la variable `ConnectionStrings__Productos` y el archivo appsettings utilizado. Un cambio inadvertido de conexión puede parecer un fallo de persistencia.

No encuentro un producto por una búsqueda de texto que parecía equivalente. Revisa el contenido exacto de la base y las reglas de comparación de cadenas del proveedor SQL Server. El método `Contains` no promete una comparación universal e independiente de collation.

Cierre: un catálogo MVC con estado persistente

Al acabar el recorrido esencial, debes poder describir cada operación sin confundir capas. El navegador envía un formulario; el binder construye un `ProductoFormulario`; DataAnnotations y ModelState permiten rechazar una entrada incorrecta; el controlador localiza o construye la entidad `Producto`; EF Core ejecuta la operación sobre LocalDB; y MVC redirige a una página de lectura que representa el estado resultante.

El punto de llegada no es que aparezca un mensaje diciendo «Guardado», sino que **el dato creado puede consultarse y editarse por su identificador, sobrevive al reinicio, y desaparece cuando se ejecuta una eliminación confirmada**. El alumnado que conoce EF Core debe ser capaz de conectar ese comportamiento con `Add`, cambios de propiedades, `Remove` y `SaveChangesAsync`, pero también con el contrato y las rutas MVC.

El siguiente punto, **P2.5**, utilizará una Razor Page muy pequeña dentro de la misma aplicación para comparar el método de trabajo con un controlador y una vista. No crearemos otro CRUD ni una API adicional: la finalidad es distinguir `PageModel`, `OnGet` y `OnPost` mientras reutilizamos el conocimiento que acabamos de consolidar.

Cuaderno de profundización opcional P2.4 — Reconstruir la historia de un producto

Este recorrido se realiza **después** de los pasos 2.4.3, 2.4.4 y 2.4.5; exige que sus bloques estén activados. No añade un segundo CRUD ni nuevas operaciones de base de datos. Su objetivo es comprobar que distingues los efectos de cada petición y que puedes diagnosticar una incidencia atendiendo al Id y a la persistencia. Trabaja con una base propia y conocida, tal como se explica en 2.4.1.

Primera etapa: crear con identidad verificable. Abre `/Productos/Crear` y envía «Carpeta roja» con un precio correcto. Observa a qué dirección redirige el servidor. Anota el identificador que aparece en la ruta de detalle y verifica en esa ficha el nombre y el precio. No inventes un Id esperado: depende de la base concreta y del historial de inserciones. Comprueba que el mismo Id aparece en el listado. La coincidencia entre redirección, ficha y listado es una evidencia más fuerte que el mero mensaje «Producto creado».

Segunda etapa: edición sobre la misma entidad. Sigue el enlace Editar asociado a ese Id. Cambia el precio y conserva el nombre. Antes de enviar, identifica cuál es el campo de formulario que transporta el nuevo precio y de dónde procede el identificador. Después comprueba la redirección y la ficha actualizada. El producto debería mantener su Id: queremos cambiar una fila existente, no agregar otra. Si en el listado aparecen dos «Carpeta roja» cuando esperabas una, comprueba si ejecutaste accidentalmente Crear en lugar de Editar o si ya existía un homónimo antes del ejercicio.

Tercera etapa: un intento inválido sin efecto persistente. Vuelve a Editar y sustituye temporalmente el nombre por «X», manteniendo un precio correcto. El

formulario debe mostrar la longitud inválida y conservar el dato para corregirlo. Sin embargo, al abrir una consulta nueva del detalle, la entidad debe seguir teniendo el nombre válido anterior. En esta etapa la diferencia importante es entre lo que **escribiste en un formulario rechazado** y lo que **está guardado en SQL**. Si ambos cambian, revisa el orden de `ModelState.IsValid` y `SaveChangesAsync` en el controlador.

Cuarta etapa: confirmar no es borrar. Abre la página GET de eliminación, lee el producto que se propone eliminar y vuelve al detalle sin aceptar. La entidad debe seguir disponible. Ahora regresa y confirma mediante el POST protegido. Comprueba la redirección al catálogo y prueba la URL del detalle que anotaste anteriormente: debe dejar de encontrar la entidad. Si el enlace simplemente desaparece del listado pero el detalle aún devuelve el producto, no tienes prueba de eliminación real.

Quinta etapa: reinicio con la misma conexión. Cierra Kestrel y vuelve a arrancar la aplicación sin modificar `ConnectionStrings__Productos`. Comprueba que la entidad eliminada continúa ausente y que otros productos del catálogo permanecen. Si todos los datos parecen cambiar de forma inesperada, compara primero la cadena de conexión y el nombre de la base, en lugar de borrar archivos o reinstalar LocalDB. El reinicio no debería activar una semilla que reemplace productos existentes; `EnsureCreated` se ha usado para preparar un laboratorio nuevo.

Interpretación arquitectónica. Relaciona cada etapa con los métodos de EF Core que ya conoces. En la creación se añade una entidad y se persiste; en la edición se recupera una existente, se modifican propiedades y se guardan los cambios; en la eliminación se localiza y elimina una entidad; en las consultas se proyecta el resultado a una vista. Anota en qué etapas se utiliza GET y en cuáles POST, y explica por qué la comprobación posterior es otra lectura en lugar de confiar ciegamente en la respuesta del POST.

Diagnóstico de transferencia. Si un alumno informa que la edición «ha funcionado» porque vio una pantalla sin error, plantea tres preguntas: ¿se redirigió a la ficha del mismo Id?, ¿muestra el nuevo valor tras una consulta?, ¿se conserva después de reiniciar? Si el alumno informa que el borrado «ha funcionado» porque desapareció un botón, plantea otras tres: ¿hubo un POST con token?, ¿el detalle devuelve 404?, ¿continúa ausente tras reiniciar? Las respuestas transforman una impresión visual en evidencia funcional comprensible.

Límite deliberado. No usamos este ejercicio para enseñar concurrencia avanzada, migraciones de base de datos o administración de SQL Server. Su profundidad procede de reconstruir con exactitud el ciclo que ya existe, observar los estados y comprender las razones de cada transición. Consulta la profundización teórica 2.4 para relacionar formulario, modelo de entrada, entidad y representación HTML.

Resolución guiada P2.4 — Diagnóstico de persistencia y reconstrucción de evidencias

A diferencia de los capítulos anteriores, ahora las operaciones generan efectos en la base de datos. Esta profundización no obliga a instalar ninguna herramienta adicional ni a crear nuevas clases. Utiliza las acciones, formularios y vistas que ya están en `PROYECTO/2.4/CursoWeb.Mvc` una vez activados los bloques 2.4.3, 2.4.4 y 2.4.5. El objetivo es consolidar una metodología de diagnóstico que un alumno pueda aplicar al salir del aula.

Incidencia A: «he creado un producto, pero no sé dónde está»

Comienza desde el formulario `/Productos/Crear`. Introduce un nombre distintivo que no aparezca en el catálogo y un precio dentro del rango permitido. Después de enviar, observa la dirección que muestra el navegador. La implementación del ejercicio redirige al detalle del producto recién creado; el identificador no se conoce de antemano. Anota ese Id, vuelve al listado y busca el nombre. Abre la ficha desde allí y comprueba que el Id coincide.

Si el POST devuelve una página de formulario con errores, la creación no se ha completado: lee los mensajes de `ModelState` y corrige los datos antes de atribuir el comportamiento a SQL Server. Si el formulario aparentemente se acepta pero no se produce la redirección, inspecciona la acción POST y comprueba que se ejecuta `SaveChangesAsync` antes de construir la respuesta de navegación. Si la redirección existe pero el detalle devuelve 404, revisa qué base se utilizó y qué Id se está enviando a la acción Detalle.

Un diagnóstico profesional no debe afirmar «se guardó» únicamente porque desapareció la pantalla de edición. Necesitamos una consulta posterior de la misma entidad. La ruta de detalle, su contenido y una repetición de la consulta constituyen la evidencia observable que buscamos.

Incidencia B: «después de editar veo el dato antiguo»

Abre la vista Editar del Id anterior. Cambia solo el precio y envía el formulario. El controlador debería recuperar la entidad por su identificador y aplicar los nuevos valores a esa entidad; no debe crear otra fila con un nombre semejante. Si al llegar al detalle aún aparece el precio anterior, comprueba qué valor envió el navegador y qué Id se utilizó en la ruta POST. Revisa después el orden de las instrucciones que asignan propiedades y ejecutan `SaveChangesAsync`.

No empieces por eliminar la base ni por cambiar el contexto EF Core. Una parte importante de las incidencias de edición procede de los identificadores y de la respuesta que se está consultando. Por ejemplo, una URL que apunta a otro Id puede mostrar un precio correcto para otro producto, pero eso no valida la operación que pretendías. El enlace construido desde el listado es una pista concreta para relacionar pantalla y entidad.

Como control adicional, vuelve a abrir la edición y escribe un nombre de un solo carácter. Comprueba que se muestra el error, que puedes corregirlo y que la base mantiene el nombre anterior. Esta operación negativa no debería generar una nueva fila ni sobreescribir datos. Explica por qué devolver la vista con `ModelState` inválido resulta preferible a guardar y después intentar revertir.

Incidencia C: «eliminar oculta el producto, pero la fila sigue existiendo»

La eliminación del ejercicio dispone de dos pasos distinguibles. Primero se abre la confirmación mediante GET. En ese momento la entidad debe seguir disponible, y la vista solamente pregunta si deseas continuar. Después, al enviar el POST protegido, la acción elimina la fila y redirige. Para investigar un resultado dudoso, separa explícitamente ambas peticiones y no infieras que un GET ha producido la escritura.

Antes de confirmar, guarda la URL del detalle. Tras el POST, comprueba que la consulta a esa URL ya no encuentra la entidad. Si devuelve 200 con el mismo producto, necesitas averiguar si se ejecutó realmente la acción POST, si se incluyó el token antiforgery, si el Id era el correcto y si se llamó a `SaveChangesAsync`. Ocultar un enlace en el HTML no equivale a borrar un registro. Si la eliminación se confirma correctamente, un nuevo GET de detalle debería informar de la ausencia mediante la respuesta adecuada.

Incidencia D: «los productos desaparecen al reiniciar»

LocalDB usa un nombre de instancia y una base de datos identificados mediante cadena de conexión. Si ejecutamos Kestrel desde una consola con una variable de entorno, pero abrimos después la aplicación desde Visual Studio sin esa variable, es posible utilizar una conexión diferente. Antes de concluir que la persistencia se ha perdido, compara los valores efectivos de configuración en ambas formas de arranque. El laboratorio propone una base aislada precisamente para que los experimentos del curso no alteren datos ajenos.

Recuerda también la semilla inicial. `EnsureCreated` prepara el esquema si es necesario, pero la lógica del proyecto no está pensada para borrar y volver a insertar todos los registros cada vez que se inicia la aplicación. Que aparezcan los tres productos originales en una base nueva no significa que un registro creado anteriormente deba reaparecer después de haberlo eliminado de forma correcta. Los cambios válidos deberían mantenerse al reiniciar con la misma cadena; los cambios rechazados por validación no deberían aparecer nunca.

Construir una pequeña historia verificable

Para consolidar, redacta una historia de un único producto utilizando su Id real: cuándo se creó, qué valor tenía inicialmente, qué dato se editó, qué intento inválido se rechazó, cuándo se borró y qué devolvió su URL tras reiniciar. Adjunta a cada etapa un comportamiento observado en navegador: dirección, texto de pantalla o código HTTP cuando lo hayas comprobado. Este documento personal no se guarda en el repositorio y no requiere crear otra aplicación.

La respuesta excelente no es una narración genérica de «crear, leer, actualizar y eliminar». Debe indicar **qué solicitud produjo cada cambio** y **qué lectura posterior confirmó la nueva situación**. Esa diferencia convierte una sucesión de clics en una prueba comprensible de persistencia. En un proyecto profesional podrías automatizarla, pero para este nivel introductorio basta con comprender y reproducir la secuencia desde Visual Studio y el navegador.

Si tras todos los pasos el catálogo funciona, no extrapoles que la aplicación ya posee autorización de usuarios, control de concurrencia o una estrategia de migraciones. Esas capas tienen objetivos propios. Aquí habrás demostrado correctamente la integración de MVC, validación y EF Core bajo unas condiciones de laboratorio explícitas.

P2.5. Comparar MVC y Razor Pages dentro de la misma aplicación

Objetivo y preparación

En P2.4 terminaste un catálogo MVC que consulta y modifica productos con EF Core. Ahora observarás una manera diferente de organizar **una página con un GET y un POST**, sin reconstruir el catálogo ni crear una segunda aplicación. La demostración utilizará el mismo proceso web, el mismo layout y dos archivos ya preparados: `Pages/Comparacion.cshtml` y `Pages/Comparacion.cshtml.cs`.

Abre este proyecto: `M02_FORMULARIOS_RAZOR/PROYECTO/2.5/CursoWeb.Mvc/CursoWeb.Mvc.csproj`. El estado inicial contiene `OnGet` activo, `OnPost` comentado, dos anotaciones de validación comentadas, un handler de reto comentado y enlaces de navegación preparados. **No debes crear ni eliminar esos archivos.**

El itinerario esencial (observación GET, activación POST y navegación) está diseñado para los **15 minutos prácticos** del temario. Los pasos de validación, aplicación autónoma y reto son una profundización posterior. La teoría correspondiente es el capítulo 2.5 de `M02_TEORIA.md`, apoyado por los capítulos 2.1–2.4.

Este proyecto hereda el arranque de P2.4 y **necesita LocalDB disponible**, incluso si `/Comparacion` no consulta datos, porque `Program.cs` ejecuta `EnsureCreated()` al iniciar. Comprueba primero el entorno de P2.4. No borres ni sobrescribas la base de prácticas anterior. Si quieres aislar los datos, establece antes de arrancar una cadena a una base nueva, por ejemplo desde PowerShell:

```
$env:ConnectionStrings__Productos = "Server=(localdb)
\MSSQLLocalDB;Database=CursoWebP25Alumno;Trusted_Connection=True;TrustServerCertifica
te=True"
```

Después, desde la raíz del repositorio:

```
cd M02_FORMULARIOS_RAZOR/PROYECTO/2.5/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --launch-profile CursoWeb.Mvc
```

El perfil incluido anuncia `http://localhost:5091`. Si Visual Studio o Kestrel indican otro puerto, reemplázalo en las direcciones siguientes. Una variable de entorno

configurada en una consola no se propaga automáticamente a una instancia de Visual Studio iniciada por separado.

Paso 2.5.1 — Confirmar la coexistencia de MVC y Razor Pages

Objetivo. Localizar las dos familias de endpoints y comprobar que el catálogo anterior sigue operativo. **Estado inicial:** no debes activar ni eliminar código; este paso es de lectura.

Abre `M02_FORMULARIOS_RAZOR/PROYECTO/2.5/CursoWeb.Mvc/Program.cs`. Localiza las marcas `PASO 2.5.1` y observa:

```
builder.Services.AddControllersWithViews();
builder.Services.AddRazorPages();
// ...
app.MapRazorPages();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Los métodos `Add...` registran servicios y los `Map...` publican las rutas. La primera pareja permite las páginas con `@page`; la segunda conserva controladores y vistas. No agregues otra aplicación `WebApplication` ni un nuevo puerto para Razor Pages.

Ejecuta y comprueba. Abre `http://localhost:5091/Productos` y luego `http://localhost:5091/Comparacion`. Ambas direcciones deben devolver una página. La primera conserva el catálogo MVC; la segunda corresponde a la Razor Page. Si toda la aplicación falla al arrancar con una excepción de SQL Server, revisa la conexión LocalDB heredada de P2.4 antes de modificar el routing.

Por qué funciona: son endpoints distintos publicados por el mismo host. **Teoría relacionada:** «El mismo host registra MVC y Razor Pages».

Paso 2.5.2 — Seguir una petición GET hasta OnGet

Objetivo. Identificar quién prepara el mensaje de la nueva página y cómo se representa en HTML. **Estado inicial:** `OnGet` ya está activo; todavía no actives `OnPost`.

Abre `Pages/Comparacion.cshtml` y localiza sus dos primeras directivas:

```
@page "/Comparacion"
@model CursoWeb.Mvc.Pages.ComparacionModel
```

Abre ahora `Pages/Comparacion.cshtml.cs` y lee:

```
public void OnGet()
{
    Mensaje = "OnGet prepara la página sin enviar un formulario.";
}
```

En `Comparacion.cshtml`, la línea `<p id="mensaje-page">@Model.Mensaje</p>` representa ese valor. Solicita `GET /Comparacion`, actualiza la página e inspecciona el HTML. Debes encontrar el texto de `OnGet` y el elemento `mensaje-page`. En las herramientas de desarrollo del navegador puedes observar la petición GET y su respuesta.

Experimento previo al POST: el formulario ya muestra un botón, pero `OnPost` sigue comentado. Envía «Ana», observa lo ocurrido y comprueba que no aparece el mensaje `OnPost ha recibido: Ana`. **No presupongas un HTTP 405:** que no exista un handler POST explícito no garantiza ese código en Razor Pages. Vuelve después a cargar la URL mediante GET.

Por qué funciona: la directiva `@page` define el endpoint, `@model` enlaza con la clase `PageModel` y el framework selecciona `OnGet` para el GET. **Teoría relacionada:** «PageModel» y «Una solicitud, dos formas de distribuir la responsabilidad».

Paso 2.5.3 — Activar el POST y seguir el enlace de datos

Objetivo. Hacer que el mismo formulario atienda POST sin añadir un controlador MVC. **Estado inicial:** el handler `OnPost` existe pero está comentado.

Abre `Pages/Comparacion.cshtml.cs`. Busca exactamente:

```
/* PASO 2.5.3 INICIO
public IActionResult OnPost()
{
    // ... cuerpo ya preparado en el archivo ...
}
PASO 2.5.3 FIN */
```

Instrucción: elimina únicamente las dos líneas que contienen `/* PASO 2.5.3 INICIO` y `PASO 2.5.3 FIN */`, manteniendo intacto el método situado entre ellas. No pegues el bloque abreviado anterior en tu proyecto; el archivo real contiene el método completo y la comprobación de `ModelState`. Guarda y recompila.

El método activo debe terminar realizando la siguiente operación cuando la entrada es aceptable:

```
Mensaje = $"OnPost ha recibido: {Nombre.Trim()}";
return Page();
```

En `Pages/Comparacion.cshtml` ya existe un formulario como este:

```
<form method="post" asp-antiforgery="true">
    <label asp-for="Nombre"></label>
    <input asp-for="Nombre" />
    <span asp-validation-for="Nombre" class="text-danger"></span>
    <button type="submit">Enviar a OnPost</button>
</form>
```

El fragmento resume los elementos relevantes; **no sustituye** la vista completa, que incluye el resumen de validación y navegación. No elimines el atributo `asp-antiforgery`.

Ejecuta y observa. Compila, abre `/Comparacion`, escribe «Ana» y envía. La respuesta debe incluir `OnPost ha recibido: Ana`. Introduce un nombre con espacios iniciales y finales y observa que el mensaje usa `Trim()`. Si falta el texto, comprueba que se activó `OnPost`, que se recompiló, y que el navegador está enviando POST a la página.

Caso negativo de seguridad: observa el campo oculto `__RequestVerificationToken` del HTML generado. Una petición POST fabricada sin ese token debe ser rechazada, en condiciones normales, con HTTP 400 antes de ejecutarse el handler. Puedes comprobarlo en una terminal distinta:

```
curl.exe -i -X POST -d "Nombre=Ana" http://localhost:5091/Comparacion
```

No desactives antiforgery para hacer que la prueba pase. Enviar desde el formulario real y fabricar un POST sin token son experimentos diferentes: el formulario legítimo incluye el token y utiliza las cookies correspondientes.

Por qué funciona: `[BindProperty]` permite recibir `Nombre` en `PageModel`; el nombre del handler selecciona el POST y `Page()` vuelve a representar esa página, sin escribir en LocalDB. **Teoría relacionada:** «Del GET al POST» y «Antiforgery».

Paso 2.5.4 — Activar la validación de servidor

Objetivo. Recuperar el significado de DataAnnotations y `ModelState` ahora aplicado a una propiedad de página. **Estado inicial:** `OnPost` debe estar activo, mientras que las anotaciones siguientes están comentadas. Este paso es de profundización.

Abre `Pages/Comparacion.cshtml.cs`, localiza `PASO 2.5.4` y elimina el prefijo `//` de estas dos anotaciones, sin modificar la línea `[BindProperty]`:

```
[Required(ErrorMessage = "Introduce un nombre.")]
[StringLength(60, MinimumLength = 2,
    ErrorMessage = "El nombre debe tener entre 2 y 60 caracteres.")]
public string? Nombre { get; set; }
```

Compila, vuelve a `/Comparacion` y ejecuta estas tres entradas de una en una:

Entrada	Resultado que debes observar
Nombre vacío	«Introduce un nombre.» junto al campo
`X`	«El nombre debe tener entre 2 y 60 caracteres.»
`Ana`	«OnPost ha recibido: Ana»

Prueba además una cadena de más de 60 caracteres. La página debe mostrar el error de longitud y conservar la entrada para corregirla. El elemento `asp-validation-for="Nombre"` presenta los errores de esa propiedad; el resumen `ModelOnly` no los sustituye.

Por qué funciona: el binder obtiene el valor, DataAnnotations registra errores en `ModelState` y `OnPost` devuelve `Page()` antes de construir un mensaje de éxito cuando hay errores. El retorno de una página inválida con HTTP 200 no significa que el dato haya sido aceptado: debes leer el HTML devuelto. Esta práctica se centra en la validación de servidor; no necesita reinstalar los scripts de validación cliente de P2.3.

Diagnóstico: si `X` se acepta, verifica que ambas anotaciones realmente quedaron activadas; si no aparece el error, comprueba el `span` con `asp-validation-for="Nombre"`.
Teoría relacionada: «DataAnnotations y ModelState siguen significando lo mismo».

Paso 2.5.5 — Navegar desde MVC a Razor Pages y regresar

Objetivo. Crear navegación cruzada sin escribir una URL literal en el enlace nuevo.
Estado inicial: el menú MVC aún no muestra «Comparar Razor Pages»; el enlace de regreso desde la página ya está activo.

Abre `Views/Shared/_Layout.cshtml` y busca `PASO 2.5.5`. La línea siguiente está encerrada en un comentario Razor:

```
@* <a asp-page="/Comparacion">Comparar Razor Pages</a> *@
```

Elimina **solo los delimitadores de ese comentario** para dejar:

```
<a asp-page="/Comparacion">Comparar Razor Pages</a>
```

No modifiques la línea explicativa anterior. Abre `Pages/Comparacion.cshtml` y comprueba que ya existe el enlace de regreso:

```
<a asp-controller="Productos" asp-action="Index">Volver al catálogo MVC</a>
```

El archivo `Pages/_ViewStart.cshtml` establece `Layout = "/Views/Shared/_Layout.cshtml"`, por lo que ambos estilos comparten el menú.

Ejecuta y observa. Compila, visita `/Productos`, pulsa «Comparar Razor Pages» y después «Volver al catálogo MVC». El primer enlace debe llevarte a `/Comparacion`; el segundo devuelve el listado. Si el navegador muestra literalmente `asp-page` como atributo en vez de convertirlo en un `href`, comprueba `Pages/_ViewImports.cshtml`, donde se registran los Tag Helpers.

Por qué funciona: `asp-page` genera el destino de la Razor Page, mientras `asp-controller` y `asp-action` generan el destino MVC. **Teoría relacionada:** «Navegación cruzada y layout compartido».

Aplicación autónoma 2.5.6 — Representar una condición con Razor

Objetivo. Reutilizar la sintaxis `@if` de M01 para interpretar una propiedad del `PageModel`. **Estado inicial:** en `Pages/Comparacion.cshtml` hay un comentario `APLICACION 2.5.6 (ESCRIBIR)`; no existe todavía una solución activa.

Tu tarea: debajo de `<p id="mensaje-page">@Model.Mensaje</p>`, escribe una condición Razor que añada un elemento con `id="respuesta-larga"` únicamente cuando `Model.Mensaje` tenga más de 40 caracteres. No modifiques `OnGet` ni el formulario. Compila y compara el GET con un POST que envíe «Ana» y otro que envíe «Ana Martínez de la Ribera». Recuerda que el mensaje inicial de `OnGet` ya supera los 40 caracteres, por lo que el elemento aparece en GET; el resultado corto del primer POST debe ocultarlo y el segundo POST volverá a mostrarlo. La condición se aplica a `Mensaje`, no directamente a la longitud de `Nombre`.

Una posible solución para consultar después de intentarlo es:

```
@if (Model.Mensaje.Length > 40)
{
    <p id="respuesta-larga">La respuesta supera los 40 caracteres.</p>
}
```

Por qué funciona: se evalúa la longitud del mensaje ya construido por el handler; no es una regla sobre la longitud de `Nombre` ni un atributo de validación. El `id` permite comprobar el resultado inspeccionando el HTML. **Teoría relacionada:** «PageModel» y renderizado de expresiones Razor.

Reto 2.5 — Un segundo handler POST que transforma el nombre

Objetivo. Comprobar que una misma página puede atender dos destinos POST sin duplicar formularios ni controladores. **Estado inicial:** `OnPostMayusculas` está comentado en el `PageModel` y el segundo botón está comentado en el archivo Razor. Es una profundización opcional.

Primera modificación. Abre `Pages/Comparacion.cshtml.cs`, busca `RETO 2.5 INICIO` y `RETO 2.5 FIN` y elimina las dos líneas delimitadoras `/* ... */`, igual que hiciste con `OnPost`. Conserva el método completo situado dentro:

```
public IActionResult OnPostMayusculas()
{
    if (!ModelState.IsValid)
        return Page();

    if (string.IsNullOrWhiteSpace(Nombre))
    {
        ModelState.AddModelError(nameof(Nombre), "Introduce un nombre.");
        return Page();
    }

    Mensaje = $"Nombre en mayúsculas: {Nombre.Trim().ToUpperInvariant()}";
    return Page();
}
```

Segunda modificación. Abre `Pages/Comparacion.cshtml`, busca el comentario `RETO 2.5` y activa exclusivamente el botón preparado:

```
<button type="submit" asp-page-handler="Mayusculas">Mostrar en mayúsculas</button>
```

Mantén el botón «Enviar a OnPost» y el formulario. No copies el nombre del método `OnPostMayusculas` dentro de `asp-page-handler`: el sufijo del handler es `Mayusculas`.

Ejecuta y comprueba. Compila, abre `/Comparacion` y escribe «Ana». El primer botón debe seguir mostrando «OnPost ha recibido: Ana»; el segundo debe mostrar «Nombre en mayúsculas: ANA». Comprueba también que el formulario sigue incluyendo el token antiforgery.

Caso negativo: envía un nombre vacío o de un carácter después de activar las anotaciones de 2.5.4. Ninguno de los dos handlers debe presentar un resultado positivo: deben mostrarse los errores de la propiedad `Nombre`. Prueba asimismo que un POST directo sin token se rechaza. Si el botón de mayúsculas no selecciona el handler, inspecciona el `action` o `formaction` HTML que generó el Tag Helper y comprueba que se transmite `handler=Mayusculas`.

Por qué funciona: Razor Pages reconoce handlers por la convención `OnPost` más el nombre `Mayusculas`. Ambos comparten `Nombre`, `ModelState` y la política antiforgery de la página. El método alternativo transforma la representación del nombre, pero no inserta, edita ni elimina productos. **Teoría relacionada:** «Más de un POST en una página».

Cierre de P2.5 y conexión con Blazor

Vuelve a `/Productos` después de realizar las pruebas. Los nombres escritos en `/Comparacion` **no deben haber aparecido como productos nuevos**. En particular, `ComparacionModel` no utiliza `AppDbContext`, `Add`, `Remove` ni `SaveChangesAsync`. Si el catálogo cambió, investiga alguna modificación ajena a esta práctica y no la atribuyas al handler de demostración.

Al concluir debes poder describir dos recorridos completos: `GET/POST /Productos/Crear` llega a acciones de `ProductosController`, utiliza `ProductoFormulario` y, cuando la entrada es válida, puede escribir en LocalDB; `GET/POST /Comparacion` llega a `OnGet` o `OnPost` en `ComparacionModel` y representa mensajes sin persistir datos.

La diferencia esencial no es que uno use HTML y otro no: ambos generan HTML con Razor y atienden HTTP. La diferencia visible es **dónde se organiza el comportamiento de la solicitud**. En el módulo siguiente estudiaremos componentes Razor de Blazor y veremos una tercera organización, ahora con estado y eventos interactivos.

Diagnóstico final. Si la ruta no existe, revisa `@page` y `MapRazorPages`; si no arranca el host, comprueba LocalDB; si el POST no modifica el mensaje, revisa `OnPost`; si falla con HTTP 400, inspecciona antiforgery; si un nombre inválido pasa, comprueba DataAnnotations; si los mensajes no aparecen, revisa `asp-validation-for` y la respuesta generada. Cambia una sola causa posible antes de volver a compilar.